

Q1 - Modellierung

In der Q1 beschäftigen wir uns mit **Modellierung**, was bereits in der E-Phase mit UML und der Objektorientierung begonnen wurde.

In diesem Halbjahr schauen wir uns [Algorithmen](#) und [höhere Datenstrukturen \(hDS\)](#) an. Wir lernen sogenannte [Design und Creation Pattern](#) kennen, die allgemeine "Schablonen" zur Problemlösung darstellen.

Am Ende des Halbjahres wird dann noch die sogenannte [Softwaretechnik](#) angesprochen.

UML - Unified Modelling Language	4
Klassendiagramme	4
Assoziationen	4
Höhere Datenstrukturen	6
Keller (Stack)	6
Umsetzung	6
PUSH	7
POP	7
Anwendungsgebiete	8
Schlange (Queue)	8
Beispiel für Enqueue	9
Beispiel für Dequeue	9
Umsetzung der Schlange (Queue) in Java	10
Ringpuffer	11
Liste (List)	12
Einfach verkettete Liste	12
Mehrfach verkettete Liste	13
Weitere Varianten	13
Anwendung der Liste:	13
Theoretische Umsetzung der Liste (List)	13
Operation "Element Löschen" (Einfach verkettete Liste)	13
Operation "Element Einfügen" (Einfach verkettete Liste)	14
Operation "Element Löschen" (doppelt verkettete Liste)	14
Baum (Tree)	15
Spezialbäume	15
Der AVL-Baum	15
Binärbäume	15
B-Baum	15
Ausgabemöglichkeiten von Binärbäume	16

Pre-Order-Ausgabe	16
In-Order-Ausgabe	16
Post-Order-Ausgabe	16
Graphen	17
Definition	17
Anwendungsbeispiele	17
Visuelles Beispiel	18
Zusammenfassung	18
Die Datenstruktur "Keller" in UML	18
Die Datenstruktur "Keller" mit Java	18
Die Klasse Knoten	18
Die eigentliche Kellerstruktur	19
Rekursion	20
Beispiel: Die Fakultät	20
Grafische Rekursion	21
Der Pythagorasbaum	21
Der Mengerschwamm in 3D	21
Algorithmen und Algorithmendesign	22
Geschichte	22
Bedeutung	22
Komplexität	22
Design-Techniken	22
Das "Maximale Teilsummen Problem"	22
Sortieralgorithmen	23
Bubblesort	23
Bubblesort mit Arrays und FOR-Schleife	24
Weitere Sortierverfahren	25
Wichtige Begriffe bei Sortierverfahren	26
In-Place	26
Stabil	26
Textsuchalgorithmen	26
Softwaretechnik	26
Kern- und Unterstützungsprozesse	27
1. Planung	27
Anforderungserhebung	28
Lastenheft (Anforderungskatalog)	28
Pflichtenheft	28
2. Analyse	30
2.1. Auswertung der Ausgangssituation	30
2.2. Mock-Up	31
2.3. Prozessanalyse / Prozessmodell	31

2.3.1 Prozessanalyse	31
2.3.2 Ein Vorgehensmodell zur Softwareentwicklung (Prozessmodell)	31
2.4 Systemanalyse Wikipedia Artikel	31
2.5 Strukturierte Analyse (SA)	32
2.6 Objektorientierte Analyse (OA)	32
3. Entwurf	33
Softwarearchitektur	33
Strukturiertes Design (SD)	33
Objektorientiertes Design (OOD)	33
Die Unified Modeling Language (Vereinheitlichte Modellierungssprache, kurz UML)	33
Fundamental Modeling Concepts (FMC)	34
4. Programmierung	34
Allgemein	34
Programmierung	35
5. Validierung und Verifikation	36
Teststufen	37
Komponententest (auch Modultest oder Unittest)	37
Integrationstest (auch Interaktionstest)	37
Systemtest	37
Abnahmetest (auch Verfahrenstest, Akzeptanztest oder User Acceptance Test (UAT))	37
6. Anforderungsmanagement	38
7. Projektmanagement	39
8. Qualitätsmanagement	39
Capability Maturity Model	40
Spice (Norm)	41
Prozess-Dimension	41
Reifegrad-Dimension	41
Incident Management	42
Problem-Management	43
Softwaremetrik	44
Ordnung von Softwaremetriken	44
Statische Analyse	46
Software-Ergonomie	46
9. Konfigurationsmanagement	47
9.1 Versionsverwaltung	47
9.2 Veränderungsmanagement	48
9.3 Release Management	48
9.4 Application Management	48
10. Softwareeinführung	48
10.1 Big Bang	48
10.2 Iterative Einführung	49

11. Dokumentation	49
11.1 Projektdokumentation	49
11.2 Softwaredokumentation	49
11.2.1 Benutzerdokumentation	49
11.2.2 Programmiererdokumentation	49
11.2.2.1 "Äußere" Dokumentation	50
11.2.2.2 Innerhalb des Quellcodes	50
11.2.3 Methodendokumentation	50
11.2.3.1 Javadoc	50
11.3 Testdokumentation	50
Anhang	51
Techniken bei der Nutzung von Zeigern/Objektreferenzen in Java	51
Tausch von Elementen	51
Weitere Beispielaufgaben zur Rekursion	52
Beispiel: Reverse	52
Beispiel: Fibonacci-Folge	52
Fibonacci-Folge mit Schleife	52
Vergleich mit der rekursiven Lösung	53
Rekursion: Die Türme von Hanoi	54

UML - Unified Modelling Language

Die *Unified Modelling Language* dient der Visualisierung von Klassen und deren Beziehungen untereinander.

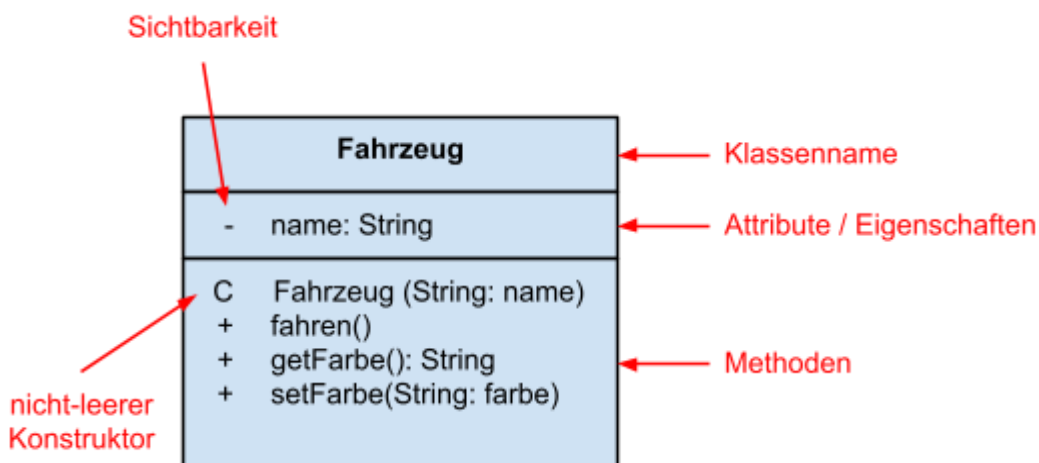


Klassendiagramme

Ein **Klassendiagramm** besteht aus dem *Klassennamen*, den *Attributen/Eigenschaften* sowie den *Methoden*.

Die **Attribute/Eigenschaften** und Methoden beinhalten neben den Bezeichnern noch die **Sichtbarkeiten** (+ oder -, public oder private) sowie den **Datentyp**.

Methoden können neben der **Sichtbarkeit** noch **Parameter** und **Rückgabetypen** enthalten. (bspw.: `verarbeite(String: text): String` bedeutet, dass die Methode `verarbeite` einen `String` (`text`) übergeben bekommt und einen `String` zurückgibt.



Der **Konstruktor** wird mit zu den Methoden geschrieben und mit einem **C** gekennzeichnet. Der Konstruktor wird immer dann aufgerufen, wenn man ein neues Objekt der Klasse erzeugt:

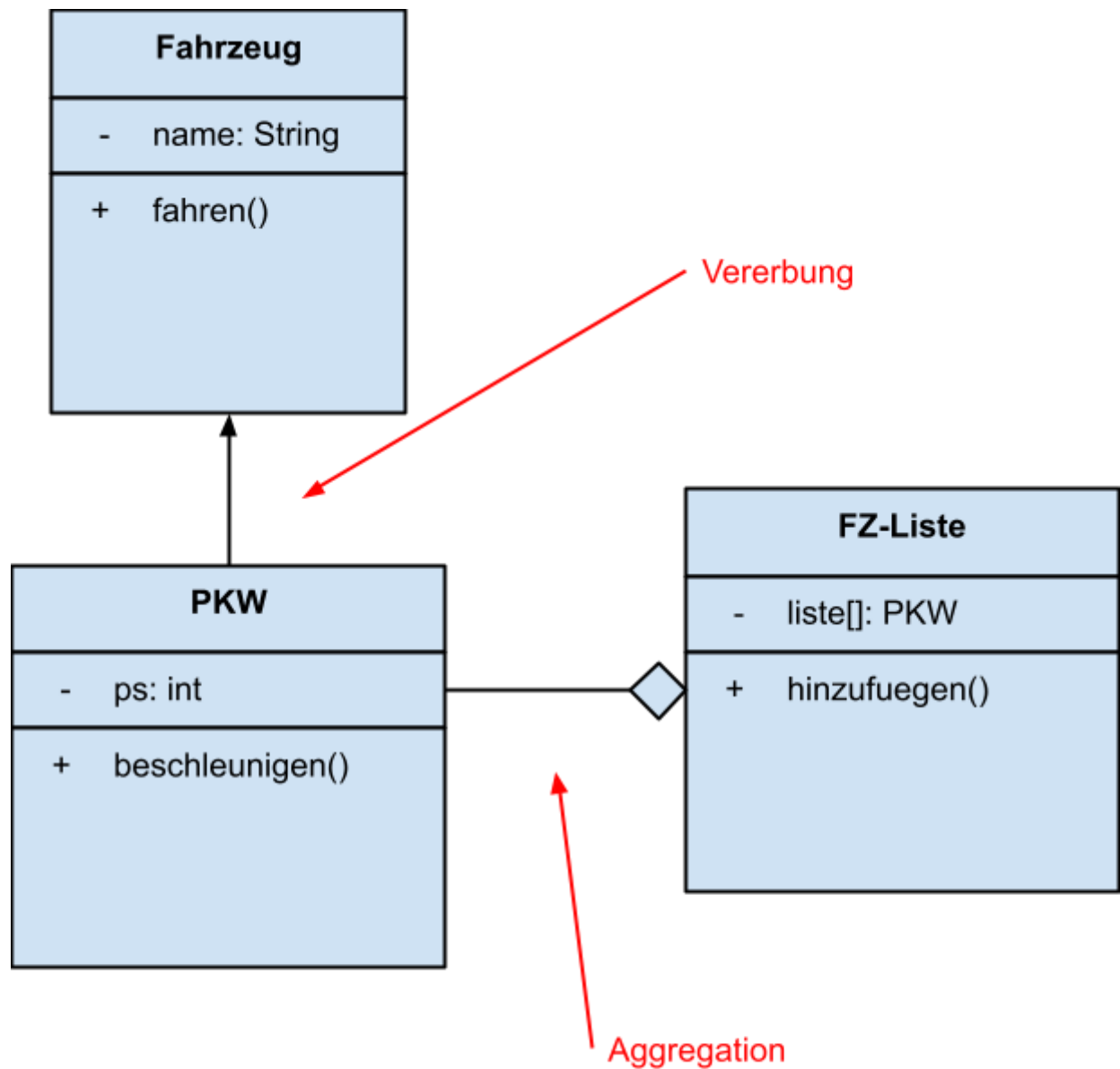
```
Fahrzeug kfz = new Fahrzeug("Audi");
```

Assoziationen

Zwischen Klassen können Beziehungen bestehen.

Es gibt die **Vererbung**. D.h., dass die erbende Klasse (hier: PKW) alle Attribute und Methoden der Oberklasse erbt.

Daneben gibt es noch die **Assoziationen**. Im Beispiel unten haben wir eine **Aggregation**. Das bedeutet, dass die Klasse `FZ-Liste` ein Attribut besitzt, welches aus Typen der anderen Klasse besteht (hier: PKW). Weiterhin gibt es noch die **Komposition**.

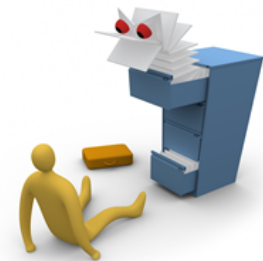


Höhere Datenstrukturen

Sie haben in der E-Phase die sogenannten primitiven Datentypen kennengelernt, wie z.B. Integer, Double oder Strings oder auch Arrays. Auf diesen primitiven Datentypen zusammen mit speziellen Creation Pattern bauen die höheren Datenstrukturen auf. Mit ihnen soll es möglich sein, die Beschränkungen (wie z.B. Speichervermögen) aufzuheben und so ganz flexible, effiziente DS zu erzeugen.

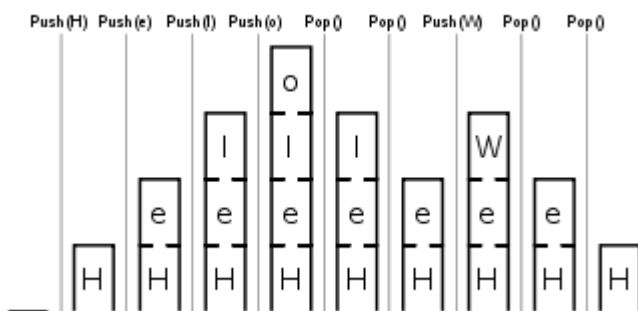
Die wesentlichen Grundtypen der hDS sind:

1. Keller (Stack)
2. Schlange (Queue)
3. Liste (List)
4. Baum (Tree)
5. (Graphen (graph))



Keller (Stack)

Ein Keller bzw. ein Stack ist eine höhere Datenstruktur, bei der mehrere Elemente übereinander abgelegt werden können (push = einkellern). Diese Elemente können nur von oben wieder ausgelesen (peek = nachsehen) oder heruntergenommen werden (pop = auskellern). Das heißt, Elemente, die in der Mitte abgelegt sind, können nur erreicht werden, indem man die oberen Elemente herunter nimmt (Nach dem Prinzip "first in-last out", kurz: FILO, manchmal ließt man auch das "last in-first out"-Prinzip, kurz: LIFO, was dasselbe bedeutet). Ein Entfernen oder Einfügen eines Elements, das im unteren Teil des Stapels liegt, ist nicht möglich.



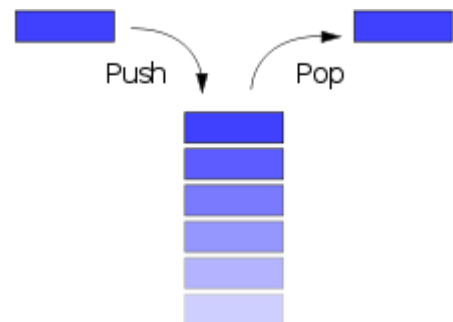
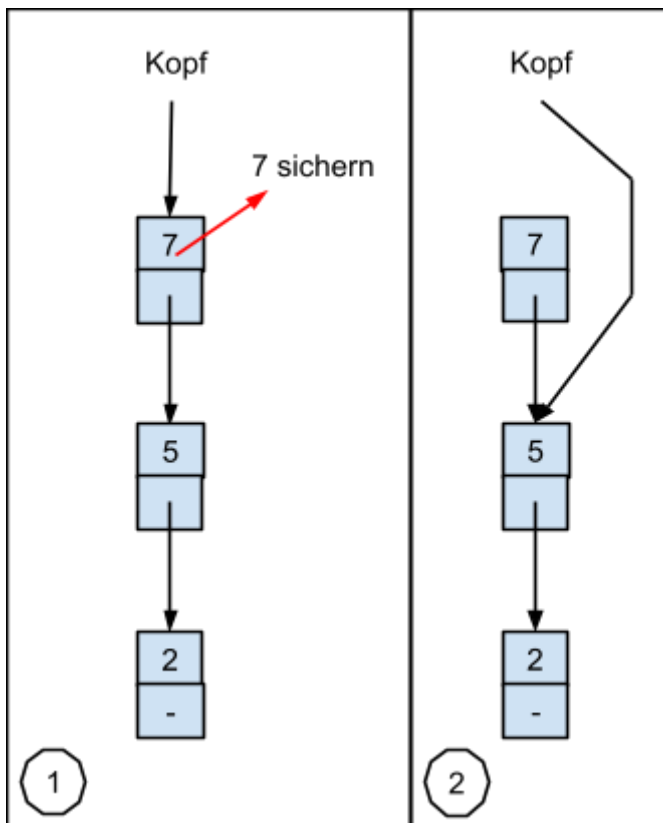
Beispiel:

Dieses Verfahren hilft z.B. bei der Fehlersuche, da man die Reihenfolge der abgelegten Elemente zurückverfolgen und somit Fehler eindeutig zugeordnet werden können.

Diese Technik wird bei der Automatentheorie (→ Q3, Stichwort: Kellerautomaten) verwendet!

Umsetzung

Die gewöhnliche Struktur eines Kellers ist die einer Liste. Das heißt, jedes Element hat einen Verweis auf das nächsttiefere. Es gibt einen Verweis, Kopf genannt, der auf das oberste Element zeigt, da man ansonsten keinen Zugriff auf den Keller hätte.



Anwendungsgebiete

In Mikroprozessoren gibt es oft ein spezielles Register, den Stackpointer (Stapelzeiger). Dieses Register enthält eine Speicheradresse, die auf den aktuellen Stapeleintrag (des aktuellen Prozesses) zeigt. Viele Befehle/Operationen des Mikroprozessors nutzen diesen Stapeleintrag.

In Multitasking-Systemen gibt es für jeden Prozess und innerhalb der Prozesse, jeden Thread einen eigenen Stapelspeicher. Beim Umschalten zwischen Prozessen bzw. Threads wird neben anderen Registern auch der jeweilige Stapelzeiger gespeichert und geladen.

Um Fehler in der Benutzung des Stacks durch einen „Unterlauf“ des Stapelzeigers aufzudecken, legen manche Betriebssysteme wie beispielsweise OSEK-OS als untersten Wert im Stapel die Sprungadresse einer Fehlerbehandlungsroutine ab. Holt der Prozessor durch einen Fehler in der Aufrufverschachtelung diese Adresse vom Stapel, kann ggf. noch auf den Ablauffehler reagiert werden.

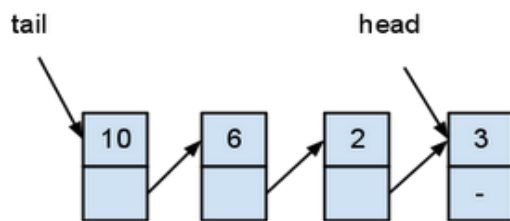
Schlange (Queue)

Eine Schlange kann eine beliebige Menge an Objekten aufnehmen und gibt diese in der Reihenfolge ihres Einfügens wieder zurück („first in-first out“-Prinzip, kurz: FIFO), vergleichbar mit einer Schlange im Supermarkt.

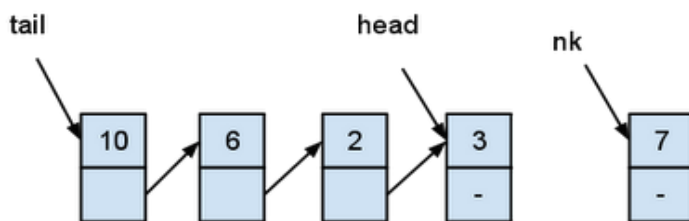
Meist verwendet man die Schlange bei Druckaufträgen oder zur Datenübergabe zwischen

asynchronen Prozessen. Es wird auch als Ringpuffer implementiert, wobei beim Ringpuffer die Menge begrenzt ist.

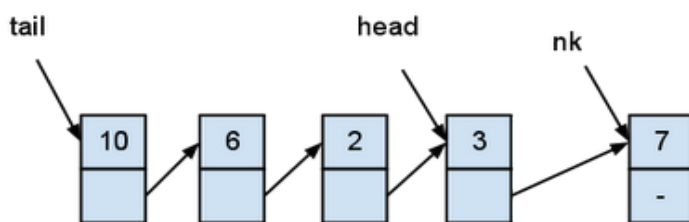
Beispiel für Enqueue



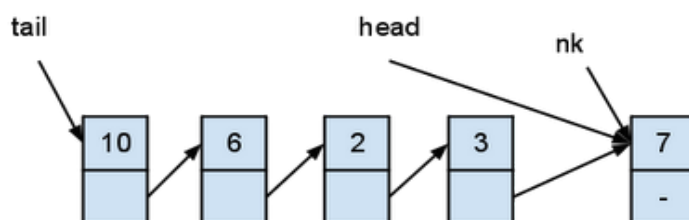
am Head anfügen:



```
Knoten nk = new Knoten();  
nk.value = 7;
```

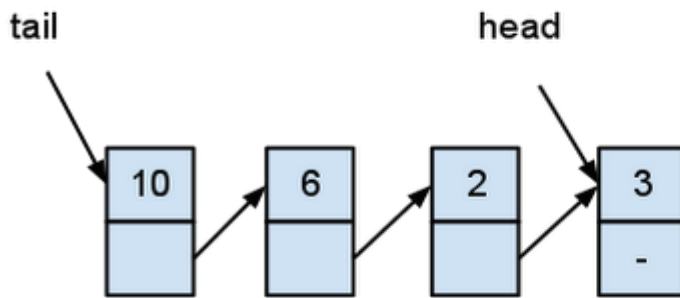


```
head.next = nk;
```

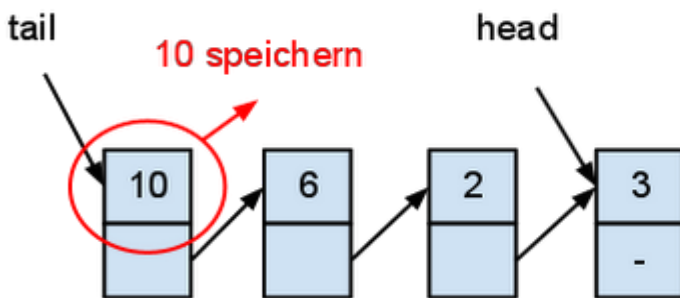


```
head = nk;
```

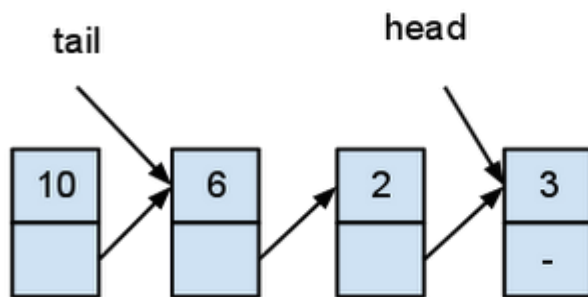
Beispiel für Dequeue



den Tail löschen:



`int zahl = tail.value;`



`tail = tail.next;`

Umsetzung der Schlange (Queue) in Java

```
public class Schlange{
    private Knoten head = null;
    private Knoten tail = null;

    public void enqueue(int newValue){
        Knoten nk = new Knoten();
        nk.value = newValue;

        if(isEmpty()){
            tail=nk;
        }

        if(head != null){
            head.next=nk;
            head=nk;
        }
        else{
```

```

        head = nk;
    }

}

public int dequeue() {
    if (isEmpty()) {
        return -1;
    } else {
        int temp = tail.value;
        tail = tail.next;
        return temp;
    }
}

public boolean isEmpty() {
    if (head == null || tail == null) {
        return true;
    } else {
        return false;
    }
}
}

```

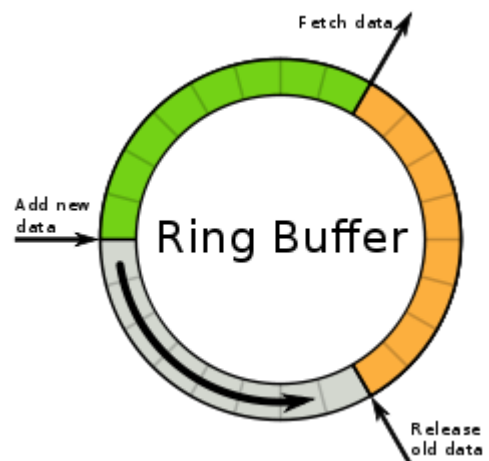
Ringpuffer

Warteschlangen sind häufig als Ringpuffer (eine "zum Kreis gebogene" Schlange) mit je einem Zeiger auf Anfang (In-Pointer) und Ende (Out-Pointer) implementiert. Dabei zeigt der In-Pointer auf das erste freie Element im Array, das den Ringpuffer repräsentiert, und der Out-Pointer auf das erste belegte Element in dem Array.

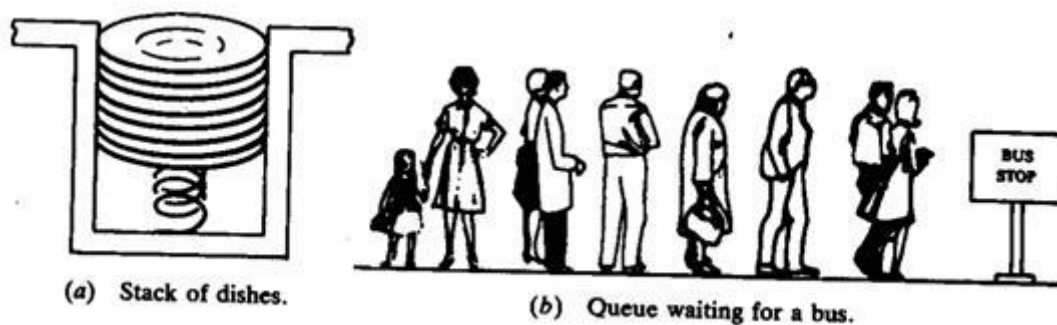
Im Unterschied zum Array werden die ältesten Inhalte überschrieben, wenn der Puffer voll ist und weitere Elemente in den Ringpuffer abgelegt werden.

Auch der Flugschreiber im Flugzeug ist in der Regel ein Ringpuffer, sodass bei einem Absturz auch nur die letzten Minuten der aufgezeichneten Messwerte vorhanden sind.

Die zur Interprozesskommunikation verwendete Pipe ist eine der wichtigsten Anwendungen für Warteschlangen. Außerdem wird es auch verwendet um langsame externe Geräte zu entkoppeln, wie zum Beispiel einen Drucker. Warteschlangen werden außerdem häufig zur Datenübergabe zwischen asynchronen Prozessen in Verteilten Systemen verwendet, wenn also Daten vor ihrer Weiterverarbeitung gepuffert werden müssen. Eine Warteschlange kann auch zum Parallelisieren verwendet werden. Dies kann man sich wie bei einem Postamt vorstellen, bei dem es mehrere



Schalter für eine Warteschlange gibt. So können Aufgaben eingestellt werden, die später parallel abgearbeitet werden.



Liste (List)

Die Liste ist eine Datenstruktur, in der eine vorher nicht bestimmte Anzahl von in Beziehung zueinander stehenden Objekten gespeichert werden kann. Die einzelnen Speicherzellen müssen nicht nacheinander im Speicher abgelegt werden, sondern die Speicherorte werden absolut referenziert.

In einer Liste zeigt jedes Objekt mit einem Zeiger auf ein anderes Objekt mit dem es in einer Beziehung steht. Diese Konstruktion aus Objekt und Zeiger nennt man einen Knoten.

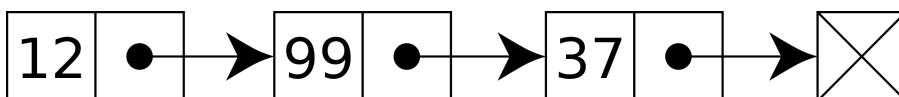
Ein Beispiel: Eine Liste bestehend aus Objekt A, B und C.

Der Zeiger des Knotens A zeigt auf das Objekt B und dessen Zeiger zeigt wiederum auf C.

Nun ist es möglich, dass der Zeiger des Knotens C auf ein beliebiges anderes Element in der Liste zeigt, z.B. B. Diese Art einer Liste nennt man auch zyklische Liste.

Bei einer Liste ist das Einfügen und Löschen von Elementen an jedem Punkt in der Liste in konstanter Zeit möglich. Dies liegt daran, dass nur Verweise neu gesetzt werden und nicht wie bei Arrays alle nachfolgenden Elemente verschoben werden müssen. Die Liste verbraucht jedoch mehr Speicher, da die Verweise auf andere Listenelemente gespeichert werden müssen.

Einfach verkettete Liste



Neben den einzelnen Knoten gibt es einen Startzeiger, der auf das erste Element zeigt. Der Zeiger des letzten Knotens zeigt auf den Wert [null](#).

Vorteile

- Elemente können sehr schnell am Anfang der Liste eingefügt werden;
- sehr geringer Speicherbedarf.

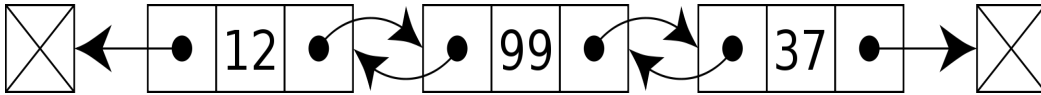
Nachteile

- Es ist aufwändig nach Daten zu suchen, Knoten einzufügen, zu löschen und die Liste zu

sortieren, da über jedes einzelne Element gegangen werden muss.

Mehrfach verkettete Liste

Bei einer mehrfach verketteten Liste hat jedes Element sowohl einen Zeiger auf das nächste, als auch auf das vorherige Element.



Der Vorgänger-Zeiger des ersten und der Nachfolger-Zeiger des letzten Elementes zeigen auf den Wert [NULL](#). Dieses besondere Element dient zum Auffinden des Anfangs und des Endes einer doppelt verketteten Liste.

Vorteile:

Diese Liste ermöglicht ein schnelles Einfügen und Löschen von Elementen und somit eine effiziente Sortierung.

Nachteile:

Allerdings benötigt diese Art der Liste mehr Speicherbedarf aufgrund der zusätzlichen Zeiger.

Weitere Varianten

- [Skip-Liste](#)

Anwendung der Liste:

In der objektorientierten Programmierung kann, bedingt durch die interne Datenstruktur, Sortierung, sortiertes Einfügen, Entfernen des größten Elementes, etc. angeboten werden.

[Weitere Informationen](#)

Theoretische Umsetzung der Liste (List)

Der Vorteil der Datenstruktur Liste besteht darin, dass man Elemente auch mittendrin einfügen und löschen kann. Die Operationen für die einfache und die doppelt verkettete Liste unterscheiden sich nicht grundlegend.

Operation "Element Löschen" (Einfach verkettete Liste)

Man geht in der Liste so lange vorwärts, bis man ein Element **vor** dem Element angekommen ist, das man löschen möchte. Man erstellt einen Hilfszeiger hz, der auf dieses Element zeigt. Dann setzt man das nächste Element des Hilfszeigers auf das übernächste Element. Dies könnte im Code etwa wie folgt aussehen:

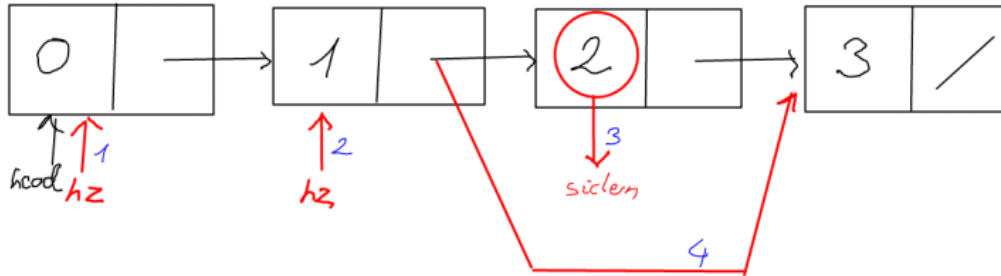
```
Knoten hz = new Knoten ();
hz = head;

for (int i=0; i< index-1; i++){
```

```

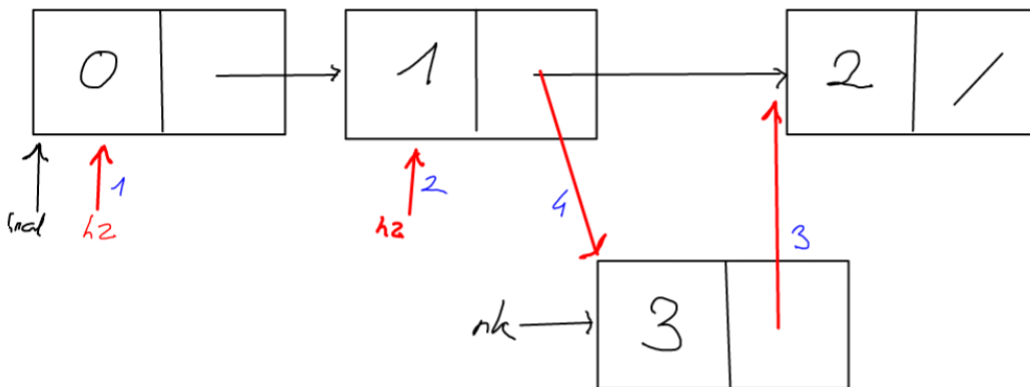
    hz = hz.next;
}
int wert = hz.next.value; // Wert speichern
hz.next = hz.next.next;

```



Operation "Element Einfügen" (Einfach verkettete Liste)

1. Zeiger hz = new Zeiger(); hz = head;
2. Iterieren bis an die i-1. Stelle (mittels Schleife)
3. nk.next = hz.next;
4. hz.next = nk;



Operation "Element Löschen" (doppelt verkettete Liste)

Man iteriert in der Liste bis ein Element vor das Element, das man löschen möchte. Man erstellt einen Hilfszeiger hz, der auf dieses Element zeigt. Dann setzt man das nächste Element des Hilfszeigers auf das übernächste Element. Zusätzlich muss man den Zeiger des Elements nach dem Gelöschten mit hz gleichsetzen. Dies könnte im Code etwa wie folgt aussehen:

```

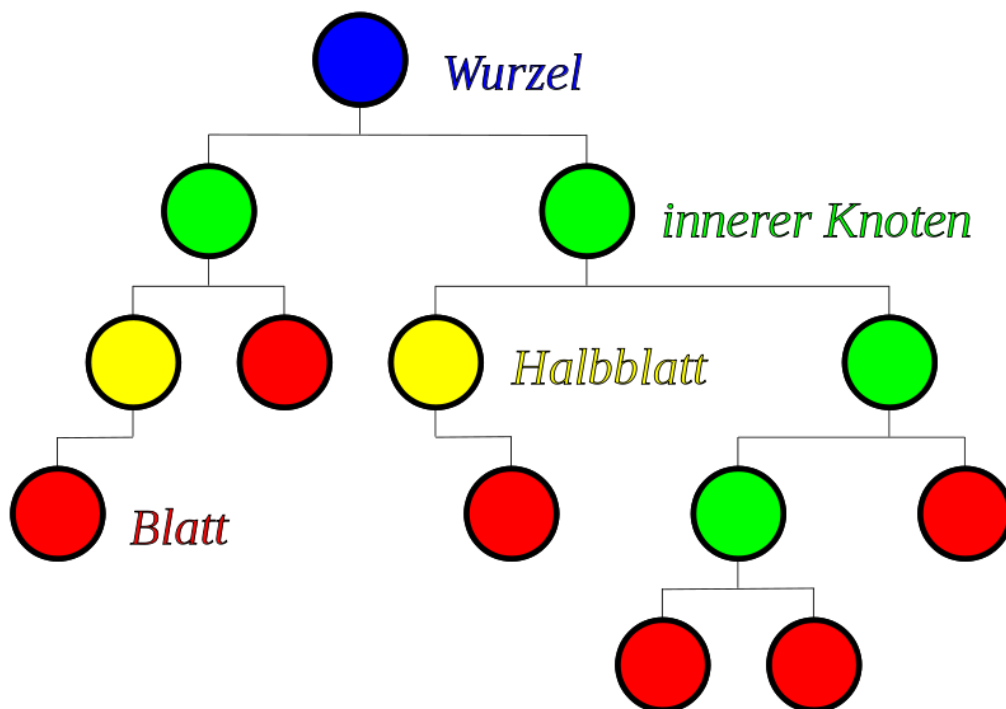
Knoten hz = new Knoten ();
for (int i=0; i< index-1; i++){

```

```
    hz =hz.next; // bis Element vor zu Löschem erreicht
}
hz.next = hz.next.next;
hz.next.prev = hz;
```

Baum (Tree)

Der Baum ist eine höhere Datenstruktur, bei der Wurzeln (Knoten) die lineare Datenstruktur der Liste aufbrechen und eine Verzweigung stattfindet. Es ist unidirektional, es kann also nur von der Wurzel nach unten gelesen werden. Es besteht oben aus einer Wurzel und verzweigt sich nach unten hin über die Kanten in Knoten. Wenn sich ein Knoten nicht weiter verzweigt, nennt man ihn Blatt.



Spezialbäume

Neben einem ganz normalen Baum wie oben beschrieben, gibt es noch eine Vielzahl von Spezialbäumen.

Der AVL-Baum

Bei [AVL-Bäumen](#) darf der Höhenunterschied der beiden Teilbäume höchstens 1 sein.

Binärbäume

An Knotenpunkten darf es maximal zwei Kindknoten, also nur zwei Objekte, geben.

Üblicherweise sind die Elemente der Knoten im linken Teilbaum kleiner, die im rechten größer als das aktuelle Wurzelement.

B-Baum

Der B-Baum wird häufig für Datenbanken eingesetzt. Es ist ein vollständig balancierter Baum, der

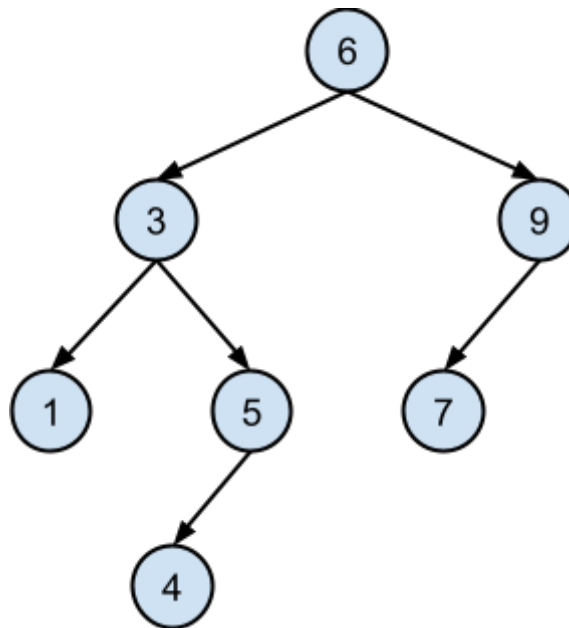
Dateien nach Schlüsseln bestimmt. Beim B-Baum kann die Höhe der Teilbäume bestimmt werden. Des Weiteren kann man einfügen, suchen und löschen.

Ausgabemöglichkeiten von Binärbäume

Bei Binärbäumen gibt es drei Möglichkeiten, die Inhalte der Knoten auszugeben: In-Order, Pre-Order und Post-Order.

Man betrachtet bei allen drei Varianten jeweils den aktuellen Knoten, den linken sowie den rechten Teilbaum. Je nach Variante in einer anderen Reihenfolge.

Dazu ein Beispiel. Gegeben ist folgender Binärbaum:



Pre-Order-Ausgabe

Dann wäre die Pre-Order-Ausgabe nach folgender Regel durchzuführen:

1. Der aktuelle Knoten.
2. Der linke Teilbaum.
3. Der rechte Teilbaum.

Die Ausgabe des obigen Baum wäre dann: 6, 3, 1, 5, 4, 9, 7.

In-Order-Ausgabe

Dann wäre die In-Order-Ausgabe nach folgender Regel durchzuführen:

1. Der linke Teilbaum.
2. Der aktuelle Knoten.
3. Der rechte Teilbaum.

Die Ausgabe des obigen Baum wäre dann: 1, 3, 4, 5, 6, 7, 9.

Post-Order-Ausgabe

Dann wäre die Post-Order-Ausgabe nach folgender Regel durchzuführen:

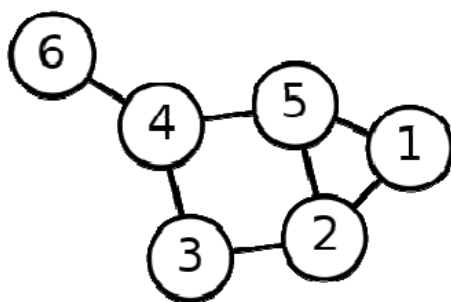
1. Der linke Teilbaum.
2. Der rechte Teilbaum.
3. Der aktuelle Knoten.

Die Ausgabe des obigen Baum wäre dann: 1, 4, 5, 3, 7, 9, 6.

Graphen

Definition

- Graph G ist ein Paar zweier Mengen $G = V, E$
 - V = Menge von n unterschiedlichen Knoten
 - E = Menge von m Kanten



$$V = \{1, 2, 3, 4, 5, 6\}$$

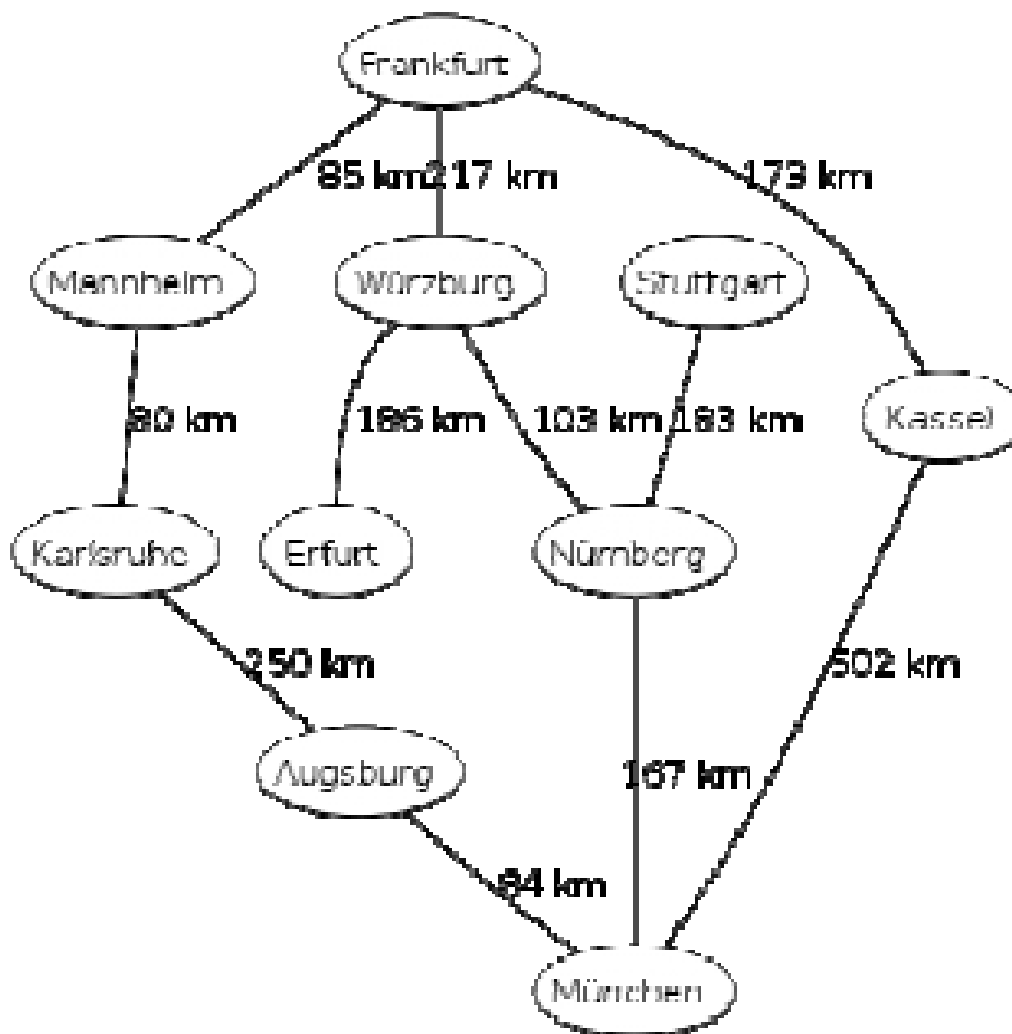
$$E = \{ (1, 2), (1, 5), (2, 3), (2, 5), (3, 4), (4, 5), (4, 6) \}$$

- Es gibt gerichtete und ungerichtete Graphen
- Es gibt Mehrfachkanten (Bsp.: hin und zurück)
- Der Graph kann verschiedenste Strukturen haben z.B.: Netze, stark/schwach zusammenhängende Graphen, Bäume
- Eulerweg (Alle Kanten werden nur einmal passiert)
- Hamiltonweg (Alle Knoten werden einmal passiert)
- Hamiltonkreis (Kreis, der alle Knoten einmal passiert)
- Gerade und Ungerade Knoten;
 - wenn die Kantenanzahl an Knoten gerade ist → gerader Knoten
 - wenn die Kantenanzahl an Knoten ungerade ist → ungerader Knoten

Anwendungsbeispiele

- Routenplanung: kürzester Weg von A nach B, kürzeste Rundreise durch A, B, C...
- Planung von Fahrplänen
- Planungsprobleme A vor B, B vor D, A vor E, D vor E → A, B, C, D, E
→ Beispiel: Google Maps (Zwischenstopp)
- Rechnernetze
- Struktur von Websites
- 3D Modellierung

Visuelles Beispiel



Zusammenfassung

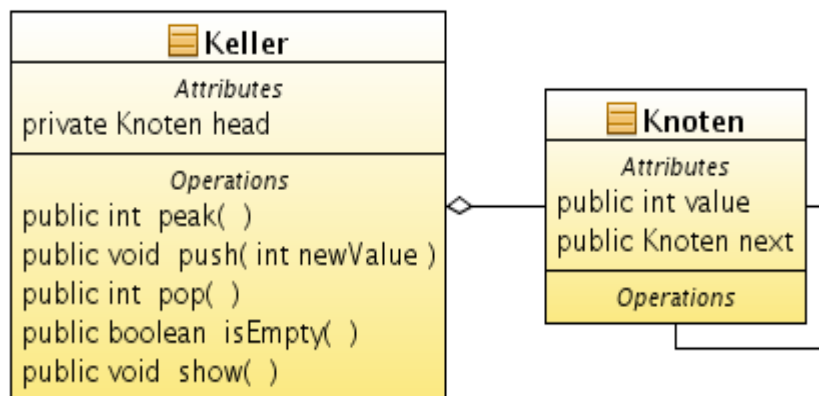
Zusammenfassend lässt sich sagen, dass alle höheren Datenstrukturen durch die Baumstruktur realisierbar sind.

Listen sind sogenannte "entartete Bäume", deren Knoten nur jeweils einen Nachfolger (Kindknoten) besitzen. Schlange und Keller sind wiederum spezielle Listen, bei denen weitere Einschränkungen gelten.

In der Implementierung realisiert man jedoch z.B. einen Keller nicht durch Anpassung des Baumes auf die Regeln des Kellers, sondern implementiert aus Effizienzgründen nur die nötigen Methoden.

Alle höheren Datenstrukturen, die wir hier betrachten, haben gemeinsam, dass sie dynamische Strukturen sind, d.h. in ihrer Größe variabel sind. Dies bedeutet auch, dass jeder Knoten irgendwo im Speicher abgelegt werden kann. Dies ist der große Unterschied zu Arrays, bei denen die zu speichernden Elemente an einem Stück im Speicher (meist RAM) geschrieben sind.

Die Datenstruktur "Keller" in UML



Die Klasse Keller beinhaltet die zusätzlichen Methoden `isEmpty()` und `show()`. Letztere gibt den Inhalt des Kellers auf der Konsole aus.

Die Datenstruktur "Keller" mit Java

Die Klasse Knoten

```
public class Knoten {
    int inhalt;
    Knoten zeiger = null;
}
```

Die eigentliche Kellerstruktur

```
public class Kellerspeicher {
    // Den Kopf des Kellers definieren:
    Knoten kopf = null;

    // Konstruktor: Den Keller initialisieren
    public Kellerspeicher(){
        kopf = null;
    }

    // Prüfen, ob Keller leer ist
    public boolean istLeer(){
        if(kopf == null){
            return true;
        } else {
            return false;
        }
    }

    // Das oberste Element anzeigen
    public Integer zeigeErstesElement(){
        if(istLeer()){
            return null;
        } else {
            return kopf.inhalt;
        }
    }
}
```

```

}

// Das oberste Element herausnehmen
public Integer pop(){
    if(istLeer()){
        return null;
    } else {
        int altesElement = kopf.inhalt; // Inhalt speichern
        kopf = kopf.zeiger; // Kopf auf nächsten Knoten umsetzen
        return altesElement; // Inhalt zurückgeben
    }
}

// Ein neues Element auf den Keller legen
public void push (int neuesElement){
    Knoten neuerKnoten = new Knoten(); // Neuen Knoten erzeugen
    neuerKnoten.inhalt = neuesElement; // Element in neuem Knoten speichern
    neuerKnoten.zeiger = kopf; // Verbindung zu bisherigem Keller herstellen
    kopf = neuerKnoten; // Kopf umsetzen
}

// Den Kellerinhalt ausgeben
public void ausgeben(){
    Knoten hz = new Knoten(); // Einen Hilfszeiger erstellen
    hz = kopf; // Hilfszeiger startet beim Kopf
    while(hz!=null){ // So lange der Hilfszeiger nicht null ist...
        System.out.println(hz.inhalt); // Inhalt ausgeben
        hz = hz.zeiger; // Hilfszeiger auf nächstes Element setzen
    }
}
}

```

Rekursion

Bei der Rekursion teilt eine Funktion ein gegebenes Problem in Teilprobleme auf, die dann durch den einfachen oder mehrfachen Aufruf derselben Funktion übergeben und bearbeitet werden. Das Problem führt so lange zu "Selbstaufrufen", sogenannten **Rekursionsschritten**, bis innerhalb aller momentan ausgeführten Funktionen der sogenannte **Abbruchfall** eintritt, weil für das jeweils gegebene Teilproblem eine Lösung vorhanden ist. Diese Lösung wird nun den vorherigen Funktionen zurückgegeben, die dann in der Lage sind, ihr Teilproblem zu lösen. Gibt es keinen Abbruchfall, so werden theoretisch innerhalb eines unendlichen Zeitraumes unendlich viele Funktionen aufgerufen. In der Praxis bricht das Programm oder das System wegen einer Speicherüberlastung zusammen. Rekursion ist im Endeffekt nichts anderes als der "Selbstaufruf" einer Funktion.

Beispiel: Die Fakultät

! → Die Fakultät ist das Produkt der Zahlen von n bis 1 (oder 0). 1! und 0! sind als 1 definiert.

5! = 5 x 4! → 120

$$\begin{aligned}
 4 \times 3! &\rightarrow 24 \\
 3 \times 2! &\rightarrow 6 \\
 2 \times 1! &\rightarrow 2 \\
 1! &\rightarrow 1
 \end{aligned}$$

```

public int fak(int n) {
    if ( n == 0 || n == 1 )
        return 1;
    else
        return n * fak(n-1);
}

```

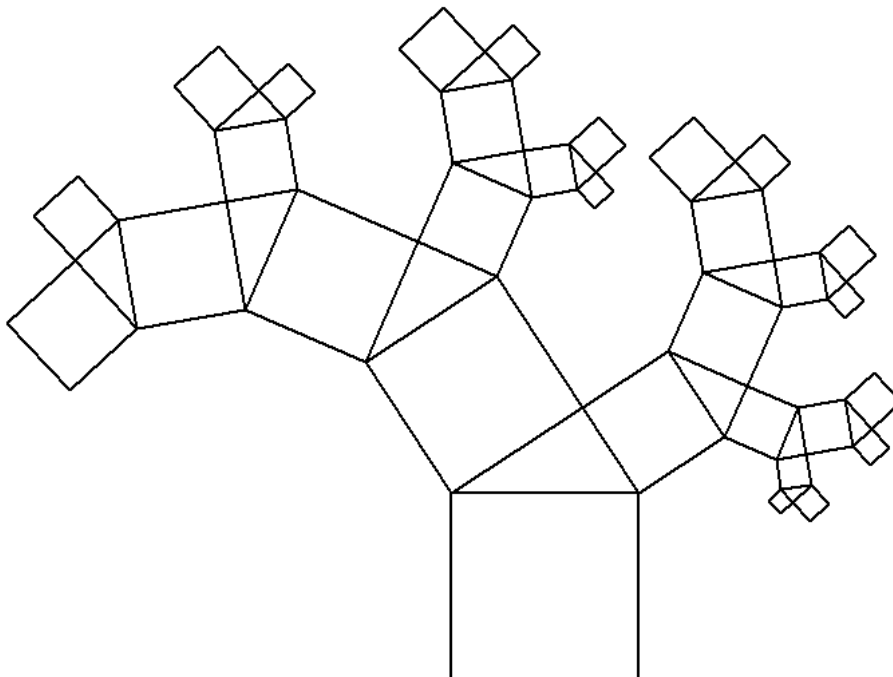
Weitere Beispiel für Aufgaben, die mit Rekursion gelöst werden können:

- harmonische Reihe, quadratische Summe
- reverse(...)
- Die Fibonacci-Folge
- [Die Türme von Hanoi](#)

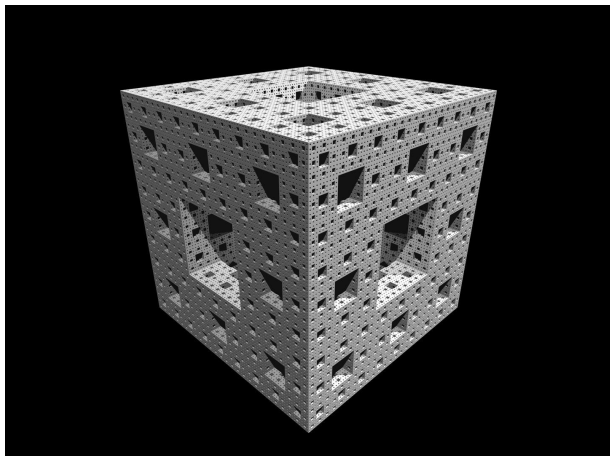
Grafische Rekursion

Rekursion in Bezug zu Grafik bringt sehr schöne Bilder zustande. Hier einige bekannte Beispiele.

Der [Pythagorasbaum](#)



Der Mengerschwamm in 3D



Algorithmen und Algorithmendesign

Geschichte

- Abwandlung oder Verballhornung des Namens von Muhammed al-Chwarizmi (etwa 783-850).
- Um 825 erschien sein Werk *über das Rechnen mit indischen Ziffern*, das mit den Worten *Dixit Algorismi* anfängt.
- Im Mittelalter wurde daraus *algorithmus*.

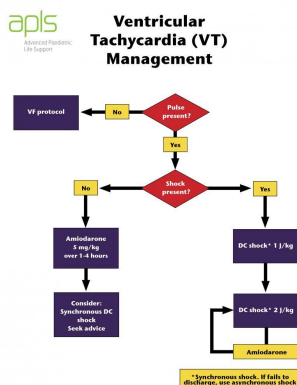


Bedeutung

Ein Algorithmus ist eine aus endlich vielen Schritten bestehende eindeutige Handlungsvorschrift zur Lösung eines Problems oder einer Klasse von Problemen.

Beispiele

- [Euklidischer Algorithmus](#)
- Algorithmus zur Berechnung der Bernoulli-Zahlen
- [Miller-Rabin-Test](#)



Komplexität

Das Laufzeitverhalten (Dauer der Verarbeitung von n-Datensätzen) wird mit dem Begriff "**Komplexität**" genauer beschrieben. Je höher die Komplexität, desto höher der Aufwand und umso länger benötigt der Algorithmus, um eine Aufgabe zu lösen.

Man gibt die Komplexität in der sogenannten [Landau-Notation](#) ($O(\dots)$) an.

Die wichtigsten **Komplexitätsklassen** sind:

- $O(x^n) \Rightarrow$ exponentielles Wachstum
 - Bsp.: Die Anzahl der Schritte vervielfacht sich, wenn n um eins erhöht wird.
- $O(n^3) \Rightarrow$ kubisches Wachstum
 - Bsp.: Die Anzahl der Schritte ver 27-facht sich, wenn sich n verdreifacht.
- $O(n^2) \Rightarrow$ quadratisches Wachstum
 - Bsp.: Die Anzahl der Schritte vervierfacht sich, wenn sich n verdoppelt.
- $O(n \cdot \log(n)) \Rightarrow$ logarithmisches oder super-lineares Wachstum
- $O(n) \Rightarrow$ lineares Wachstum
 - Bsp.: Die Anzahl der Schritte verdoppelt sich, wenn sich n verdoppelt.

Design-Techniken

Das "Maximale Teilsummen Problem"

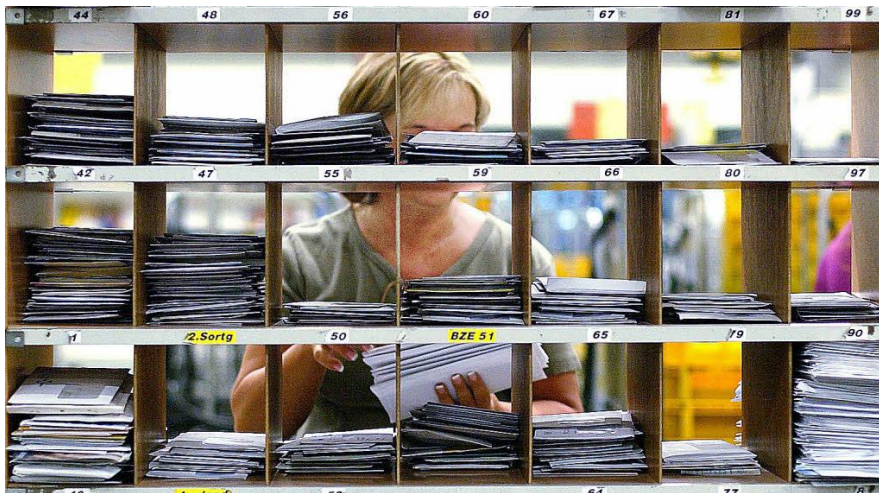
Ein schönes Problem, an dem man verschiedene Kriterien zur Optimierung von Algorithmen studieren kann, ist das "Maximale Teilsummen Problem".

Hier der Link zum Skript: [MaxPartSum.pdf](#)

Was wir aus diesen Beispielen lernen können, lässt sich in Form einiger Regeln formulieren. Gute Programmierer haben diese im Hinterkopf, wenn sie ähnliche Algorithmen zu entwickeln versuchen:

- Speichere *Zwischenergebnisse*, um mehrfache Berechnungen zu vermeiden (Alg. 2a)
- Bereite die Informationen auf und finde geeignete Datenstrukturen (Alg. 2b)
- *Divide and Conquer* ist oft gut geeignet (Alg. 3)
- *Scanning* (Abtast-) Algorithmen sind oft gut geeignet (Alg. 4)

Sortieralgorithmen



Bubblesort

Bei Bubblesort handelt es sich um einen Sortieralgorithmus mit der Komplexität $O(n^2)$. Bei Bubblesort werden immer zwei nebeneinander liegende Elemente verglichen und - falls nötig - verschoben. Dann werden die nächsten Elemente verglichen.

In Java mit einer eigenen Liste als Datenspeicher:

```
int zaehler=0;
int anzahlElemente = liste.size();
Knoten hz=head;
int i = 0;
while (i < anzahlElemente){
    while (zaehler < anzahlElemente - i){
        if( hz.value > hz.next.value){
            liste.tausche(zaehler,zaehler+1);
        }
        zaehler++;
        hz=hz.next;
    }
    i++;
}
```

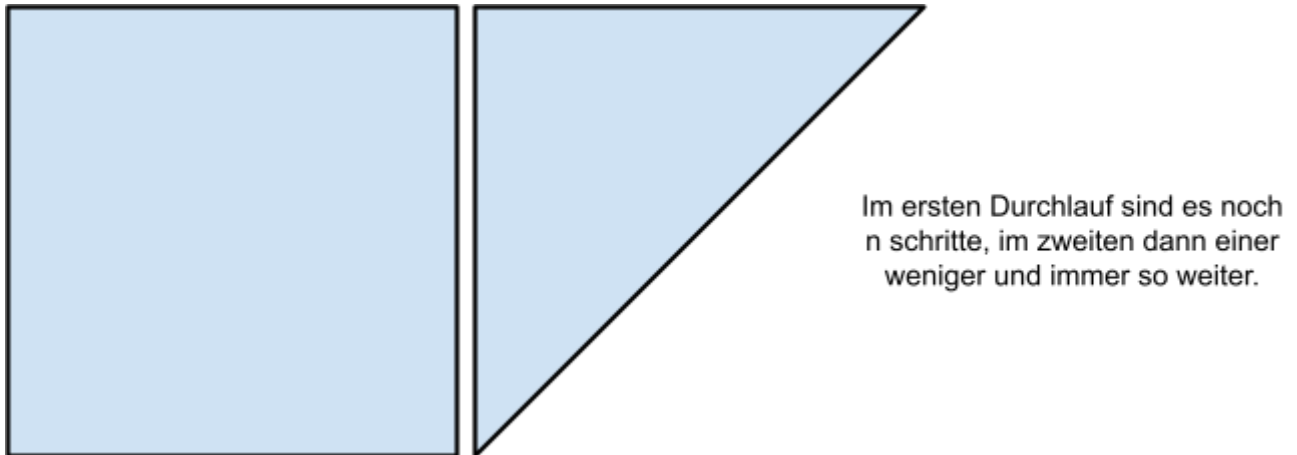
```

    zaehler = 0;
    hz = head;
    i++;
}

```

Dies ist jedoch nur der erste Durchgang. Wir brauchen $n-1$ Durchgänge, die jedesmal kürzer werden, da im ersten Durchlauf der Knoten mit dem höchsten Wert IMMER(!) an die richtige Stelle rutscht.

Sollte die kleinste Zahl ans Ende der Liste, so würde dies im ersten Zug geschehen.



Da die Schrittzahl immer abnimmt, entsteht die 2. Form, das Dreieck mit der Flächenformel ($0.5 * n^2$).

Daraus leitet sich auch die **Komplexitätsklasse** ab, die $O(n^2)$ entspricht.

Bubblesort mit Arrays und FOR-Schleife

```

public class Bubblesort {
    public static void main(String[] args) {
        int[] liste = new int[6];

        liste[0] = 7;
        liste[1] = 3;
        liste[2] = 5;
        liste[3] = 0;
        liste[4] = 1;
        liste[5] = 4;

        liste = bsort(liste);
        showListe(liste);
    }

    /* BUBBLESORT */
    private static int[] bsort(int[] liste){
        for (int i = 0; i < liste.length; i++) {
            for (int j = 0; j < liste.length - i -1; j++) {

```

```

        if(liste[j] > liste[j+1]){
            liste = tausche(liste, j, j+1);
        }
    }
    return liste;
}

/* Dreieckstausch */
private static int[] tausche(int[] liste, int i, int j) {
    System.out.println("tausche: "+liste[i]+" mit "+liste[j]);
    int temp = liste[i];
    liste[i] = liste[j];
    liste[j] = temp;
    return liste;
}

/* Liste auf der Konsole ausgeben */
private static void showListe(int[] liste){
    for(int i = 0; i < liste.length; i++) {
        System.out.print(liste[i]+" ");
    }
    System.out.println("");
}
}

```

Weitere Sortierverfahren

Es gibt eine Vielzahl von Sortierverfahren. Die bekanntesten und relevantesten für den Unterricht sind:

- Selectionsort (auch Min- oder Maxsort genannt)
- Insertionsort
- [Quicksort](#)
- [Mergesort](#)
- [Heapsort](#)

Eine Übersicht und genauere Informationen finden Sie hier:

<http://de.wikipedia.org/wiki/Sortierverfahren>

Auf folgender Seite finden Sie eine Vielzahl von Sortialgorithmen grafisch animiert und mit verschiedenen Datensätzen dargestellt:

<http://www.sorting-algorithms.com>

Wichtige Begriffe bei Sortierverfahren

In-Place

Nutzt ein Verfahren die vorhandene Datenstruktur aus, in der die zu sortierenden Elemente gespeichert sind, sagt man, es arbeitet "in-place". Arbeitet ein Verfahren nicht "in-place", wird eine weitere Datenstruktur zur Sortierung benötigt.

Stabil

Man bezeichnet ein Verfahren als *stabil*, wenn es die vorherige Ordnung gleicher Element auch nach dem Verfahren gewährleistet.

Beispiel:

- Wir haben die Zahlen: 1, 6, **2**, 4, **2**
- Sortiert haben wird dann:
 - 1, **2**, **2**, 4, 6 → **stabil**, die zwei Zweien sind in der ursprünglichen Reihenfolge geblieben
 - 1, **2**, **2**, 4, 6 → **nicht stabil**, die zwei Zweien sind nicht in der ursprünglichen Reihenfolge geblieben

Textsuchalgorithmen

In Binärbäumen kann man sehr leicht nach Zahlen suchen, die z.B. für Indizes von Datensätzen stehen. Aufwendiger und schwerer wird es in Texten. Dort kommt es auf die Reihenfolge der Worte an, die nicht verändert werden darf.

Es gibt zwei Textsuchalgorithmen, die wir näher betrachten. Siehe dazu die folgende Präsentation und die Links:

- [Präsentation](#)
- [Brute Force](#)
- [Boyer-Moore](#)

Softwaretechnik

(Quelle: Wikipedia, Stichwort: Softwaretechnik)

Die Softwaretechnik ist eine deutschsprachige Übersetzung des engl. Begriffs *software engineering* und beschäftigt sich mit der Herstellung bzw. Entwicklung von [Software](#), der Organisation und Modellierung der zugehörigen [Datenstrukturen](#) und dem Betrieb von Softwaresystemen.

Eine Definition von [Helmut Balzert](#) beschreibt das Gebiet als

„Zielorientierte Bereitstellung und systematische Verwendung von [Prinzipien](#), [Methoden](#) und [Werkzeugen](#) für die arbeitsteilige, ingenieurmäßige Entwicklung und Anwendung von umfangreichen Softwaresystemen.“ – [Lit.](#): Balzert, S.36

Softwaretechnik umfasst eine Vielzahl von Teilgebieten, die in ihrer Gesamtheit die

Softwareentwicklung begleiten. Wichtig ist auch die experimentelle Untersuchung von Softwaretechnik, um ihren praktischen Nutzen zu messen und zu verbessern.

Zur Beschreibung des „Standes der Technik“ des Fachgebiets gibt es verschiedene Ansätze, unter anderem den Guide to the [Software Engineering Body of Knowledge](#) (SWEBOK) der IEEE Computer Society.

- [Risiken bei der Softwareentwicklung - Artikel aus der Praxis, was alles schief gehen kann.](#)
- [Link zur Einfügungspräsentation](#)

Kern- und Unterstützungsprozesse

Die Softwaretechnik unterteilt sich in die sogenannten Kern- und die Unterstützungsprozesse.

Die Teile 1-4 gehören zu den Kernprozessen, die Teile 5-11 zu den **Unterstützungsprozessen**.

Bei den Kernprozessen ist eine chronologische Vorgehensweise sinnvoll, die **Unterstützungsprozesse** laufen meist während der gesamten Projektphase nebenher ab.

1. [Planung](#)

Definition:

Planung ist eine gedankliche Vorwegnahme von Handlungsschritten die zur Erreichung eines Ziels dienen. Planen unterteilt sich in abstraktes Planen und konkretes Planen.

Abstraktes Planen ist eine kognitive Fähigkeit (Kognition = Informationsumgestaltung). Beim konkreten Planen wird zusätzlich auf Informationen und Erfahrungsquellen zugegriffen.

Eigenschaften:

Merkmale :

1. Modellcharakter (Vereinfachung des Gesamtsystems)
2. Prozesscharakter (Planungsprozess läuft in mehreren Phasen ab)
3. Gestaltungscharakter (Mitgestaltung der Zukunft in einem Gebiet)
4. Informationscharakter (Information an andere Personen)

Funktionen :

1. Früherkennungsfunktion (Strukturierung der Problemstellung)
2. Orientierungsfunktion (Handlungsspielraum in die Zukunft ausweiten)
3. Koordinierungsfunktion (Verknüpfung von Abhängigkeiten)
4. Moderationsfunktion (Auflösung von Verteilungs- und Interessenkonflikten)

Die Planung lässt sich in verschiedene Schritte unterteilen.

1. Eine Anforderung wird vom Kunden erstellt (Anforderungserhebung)
2. Anforderung wird definiert (Lastenheft)
3. Lösungsvorschlag des Lastenheftes wird vom Unternehmen dem Kunden übergeben (Pflichtenheft)
4. Kostenvorschlag wird erstellt aus der Aufwandschätzung (z.B. COCOMO-Verfahren)
5. Vorgehensmodell

Anforderungserhebung

Ziel ist es, die Anforderungen des Auftraggebers an das zu entwickelnde System zu ermitteln.

Vorgehen:

- Anforderungen werden gesammelt
- Anforderungen werden analysiert (zum Verständnis)
- Modell wird erstellt
- Prüfung

Rund um diese Schritte existiert eine Verwaltung des Prozesses, das Management.

Lastenheft (Anforderungskatalog)

Im Lastenheft wird die Gesamtheit der Forderungen des Auftraggebers niedergeschrieben ("Was und Wofür"). Dieses Lastenheft wird an mehrere Auftragnehmer ausgehändigt. Dabei sollte dieses so allgemein wie möglich und einschränkend wie nötig sein um eine optimale Lösung erhalten zu können.

Pflichtenheft

Das Pflichtenheft enthält eine mögliche Lösung des Auftragnehmers für die Bearbeitung der im Lastenheft gestellten Anforderungen ("Wie und Womit"). Aus den Pflichtenheften wählt der Auftraggeber den Auftragnehmer aus.

[Hier ein Beispiel Pflichtenheft.](#)

Aufwandschätzung

Die Aufwandsschätzung wird folgendermaßen berechnet ([COCOMO-Verfahren](#)):

Der Aufwand PM (Personenmonaten) wird dann berechnet als ein Faktor m multipliziert mit einer Potenz n, der Metrikzahl. **KDSI** (1000 (K) delivered source instructions [1 KDSI = 1000 Instruktionen])

allgemein: $PM = m \cdot KDSI^n$

einfach: $PM = 2,4 \cdot KDSI^{1,05}$

mittelschwer: $PM = 3 \cdot KDSI^{1,12}$

komplex: $PM = 3,6 \cdot KDSI^{1,2}$

Beispiel : Bei 100 KDSI betragen die Personenmonate PM etwa 300 für ein einfaches Projekt, etwa 500 für ein mittelschweres Projekt und etwa 900 für ein komplexes Projekt.

Die Projektdauer wird nun folgendermaßen berechnet:

Man kann die Personenmonate jedoch nicht durch eine beliebige Anzahl von Personen teilen, um das Produkt schneller fertig zu stellen.

Beispiel: das Gebären eines Kindes – dies kann nicht durch den Einsatz von neun Frauen auf einen Monat abgekürzt werden.

Es gibt gewisse Prozesse, die sequentiell ablaufen müssen, und je mehr Menschen mit einem Projekt betraut sind, umso mehr muss in die Kommunikation investiert werden.

einfach: $TDEV = 2,5 \cdot PM^{0,38}$

mittelschwer: $TDEV = 2,5 \cdot PM^{0,35}$

komplex: $TDEV = 2,5 \cdot PM^{0,32}$

(TDEV = time to develop (Entwicklungszeit)).

Beispiel :

- Für ein **einfaches Projekt** mit 32 KDSI liefert die COCOMO-Abschätzung 91 PM und ein TDEV von 10,6 Monaten.
- Für ein **mittelschweres Projekt** mit 32 KDSI liefert die COCOMO-Abschätzung 145 PM und ein TDEV von 14 Monaten.
- Für ein **komplexes Projekt** mit 32 KDSI liefert die COCOMO-Abschätzung 230 PM und ein TDEV von 20 Monaten.

Aus der Aufwandschätzung entsteht nun die Kostenschätzung. In die Kostenschätzung fließen folgende Kriterien mit ein :

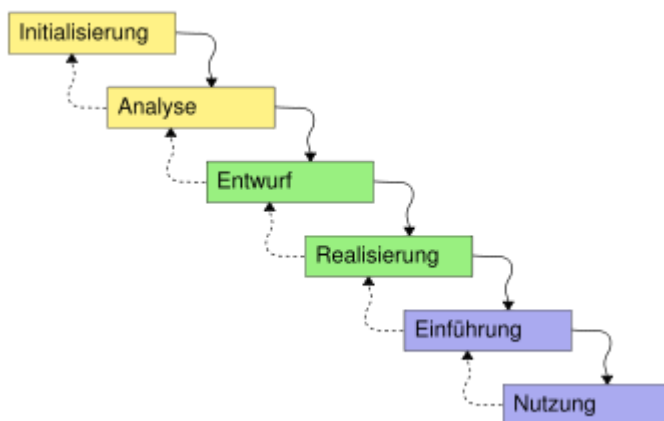
- Personalkosten (Personenaufwand , Zeitaufwand)
- Sachkosten (benötigte Ressourcen)
- kaufmännische Aufschläge (Realisierungsrisiko , Sicherheitsaufschlag für Fehleinschätzung, Vorfinanzierungskosten, Inflation)
- Kosten des Schätzverfahrens (ca. 3 % des Projektumfangs)

Vorgehensmodell zur Softwareentwicklung:

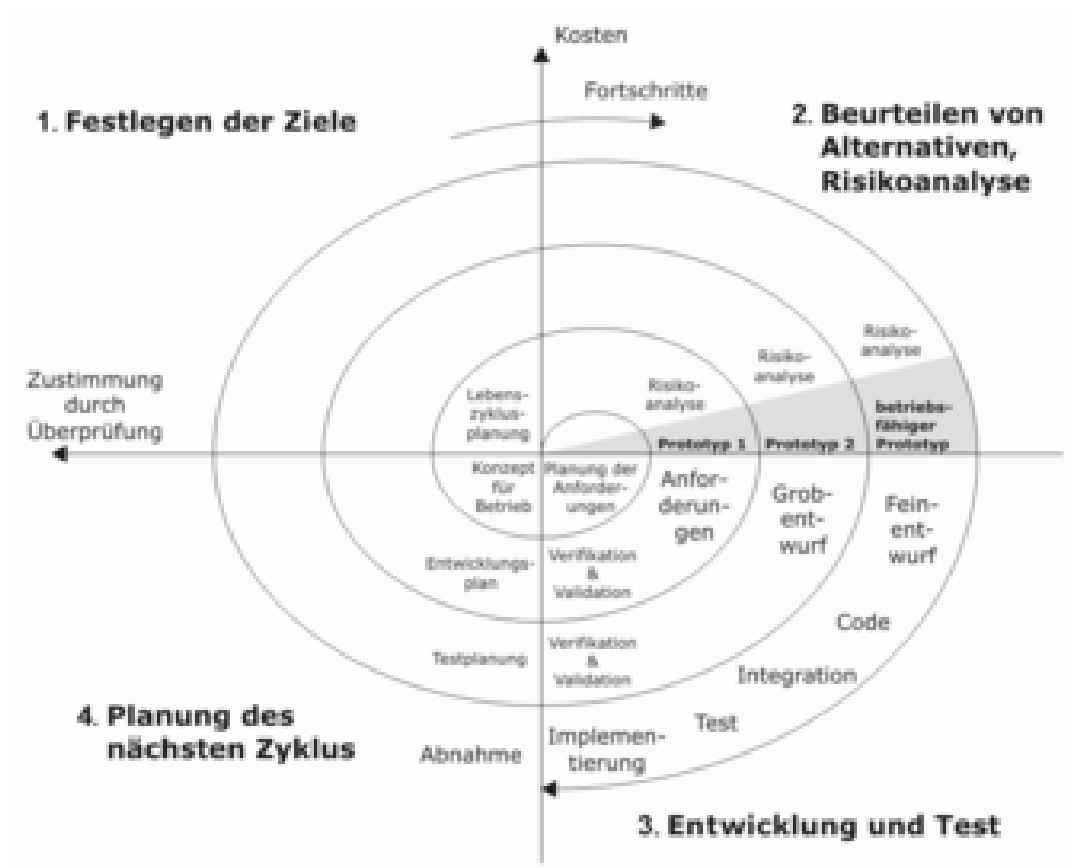
Das Vorgehensmodell dient dazu, die Softwareentwicklung übersichtlicher zu gestalten und in der Komplexität beherrschbar zu machen.

Man unterscheidet das Vorgehensmodell in zwei Arten :

Wasserfallmodell: (einmalig)



Spiralmodell: (mehrfach) → dient zur Verfeinerung der Softwarekomponenten



2. [Analyse](#)

Die Analyse ist ein wichtiger Kernprozess der Softwaretechnik.

Man kann sie in sechs Unterpunkte gliedern:

1. Auswertung
2. Mock-Up
3. Prozessanalyse / Prozessmodell
4. Systemanalyse
5. Strukturierte Analyse (SA)
6. Objektorientierte Analyse (OOA)

2.1. Auswertung der Ausgangssituation

Zunächst wird untersucht, wie der Kunde bisher gearbeitet hat. Es werden Prozesse (Abläufe) identifiziert und analysiert. Die zu verarbeitenden Daten werden gesichtet. Dazu wird mit Mitarbeitern gesprochen und es werden Arbeitsabläufe beobachtet, damit die Prozesse, die nicht immer klar voneinander getrennt erscheinen, genau spezifiziert werden können.

2.2. Mock-Up

[Zum Wikipedia Artikel](#)

Ein "Mock-Up" ist im Prinzip ein Modell eines Programms. Es wird auch als "Weg-Werf-Prototyp" bezeichnet. Hierdurch soll die Suche nach einer passenden Benutzeroberfläche, auf Anforderungen des Auftraggebers und des Konsumenten gefunden werden.

Mock-Up's werden auch als Platzhalter für noch nicht existierende Objekte verwendet, z.B. wenn eine Funktion getestet werden muss, für die spezielle noch nicht programmierte Objekte vorhanden sein müssen. Auch wird es verwendet, bei unvorhersehbarem Verhalten des Projekts. Es ist ein Modell eines noch nicht fertig gestellten Programms, was z.B. zu Präsentationszwecken dient.



Hier sieht man das Modell eine Airbus A380.

2.3. Prozessanalyse / Prozessmodell

2.3.1 Prozessanalyse

Bezeichnet die systematische Untersuchung (Analyse) von Prozessen und die Zerlegung in seine Einzelteile, um Verständnis für den Prozess zu erlangen und um Schwachstellen und Verbesserungspotentiale zu erkennen.

Es kann sich dabei um verschiedene Arten von Prozessen handeln:

- betriebswirtschaftliche Prozesse, siehe Geschäftsprozessanalyse
- soziologische Prozesse
- technische Prozesse

2.3.2 Ein Vorgehensmodell zur Softwareentwicklung (Prozessmodell)

Ist ein für die Softwareentwicklung angepasstes Vorgehensmodell bei der professionellen ("ingenieurmäßigen") Anwendungsentwicklung. Es dient dazu, die Softwareentwicklung übersichtlicher zu gestalten und in der Komplexität beherrschbar zu machen.

Bei diesem Softwaremodell gibt es allerdings auch diverse [Vor-](#) und [Nachteile](#).

[Wikipedia Artikel](#)

2.4 Systemanalyse [Wikipedia Artikel](#)

Unter Systemanalyse wird in der Informatik die erste Phase im Entwurfsprozess verstanden. Der Systemanalytiker beschreibt die für seine Fragestellung relevanten Systemelemente und deren Beziehungen zueinander (in der Regel mit einem Informationsmodell). Ziel der Systemanalyse ist es zum Beispiel die Umwelt ohne Maschine (Ist-Zustand) zu beschreiben, um ausgehend von diesem Ist-Modell eine Maschine zu planen. Das Soll-Modell zeigt, wie die Maschine aussehen soll. Durch die Unterschiede zwischen Ist- und Soll-Modell wird deutlich, was die zu konstruierende Maschine leisten soll. Im Rahmen der Systemanalyse wird nicht untersucht, wie die Maschine implementiert wird. Als *Maschine* sind in diesem Zusammenhang Hardware und Software als eine Einheit zu verstehen. Die Systemanalyse kann auch vor der Optimierung, Migration und Konvertierung von Systemen eingesetzt werden.

2.5 Strukturierte Analyse (SA)

Die Strukturierte Analyse (SA) ist eine Methode zur Erstellung einer formalen Systembeschreibung im Rahmen der Softwareentwicklung. Sie wird während der Analysephase eines Software-Projekts eingesetzt. Strukturiertes Design verfeinert die Ergebnisse der SA so weit, dass sie dann umgesetzt werden können. Sie ist eine Methode der Systemanalyse.

Das Ergebnis der Strukturierten Analyse ist ein hierarchisch gegliedertes Anforderungsdokument für Umfang und Inhalt der betrieblichen Anwendung, die in dem geplanten Softwaresystem realisiert werden soll. Die Strukturierte Analyse ist eine graphische Analysemethode, die mit Hilfe eines Top-Down-Vorgehens ein komplexes System in immer einfachere Funktionen bzw. Prozesse aufteilt und gleichzeitig eine Datenflussmodellierung durchführt. In ihrer Grundform ist die SA eine statische Analyse, die jedoch später um Methoden für dynamische Analysen erweitert wurde.

[Wikipedia Artikel](#)

2.6 Objektorientierte Analyse (OA)

In der Analyse geht es darum, die Anforderungen zu erfassen und zu beschreiben, die das zu entwickelnde Softwaresystem erfüllen soll. Stark vereinfacht ausgedrückt sucht und sammelt man in dieser Phase alle Fakten, stellt sie dar und überprüft sie. Dies geschieht oft in Form eines textuellen Pflichtenheftes oder einer Software Requirements Specification.

Ergebnis der Analyse ist ein allgemeines Produktmodell in Form eines *Objektorientierte Analyse-Modells* (OOA-Modell). Diese fachliche Beschreibung mit objektorientierten Konzepten enthält verschiedene Artefakte wie Diagramme und Darstellungen von Kontrollstrukturen, oft in UML-Elementen notiert:

- statisches Teilmodell: Attribute, Vererbungsbeziehungen zwischen Klassen, Assoziationen untereinander und Paketstrukturen
- dynamisches Teilmodell: Modellierung der Operationen und Botschaften zwischen Klassen mittels Geschäftsprozessdiagrammen, Zustandsautomaten und Szenarios

Außerdem kann das OOA-Modell enthalten:

- Klassendiagramm
- Kollaborationsdiagramm
- Sequenzdiagramm
- Entity-Relationship-Modell
- Pseudocode (implementierungsspezifische technische Details werden erst beim OOD betrachtet)
- Benutzerschnittstelle (Prototyp)
- Benutzerhandbuch (Prototyp).

[Wikipedia Artikel](#)

3. Entwurf

Softwarearchitektur

- Strukturierte oder hierarchische Anordnung der Softwarekomponenten
- Beschreibung der Softwarekomponenten untereinander
- Dokumentation von Entwicklung, Installation und Betrieb
- Kommunikationsart und -weise
- Hardware- /Softwareressourcen

Softwarearchitektur ist ein Grobgerüst und funktioniert global im Gesamtprojekt, während Softwareentwicklung lokal und sehr detailliert ist.

Sie ist der kritischste und wichtigste Punkt im Entwicklungsprozess und ist die früheste Softwaredesignentscheidung (Modifizierbarkeit, Wartbarkeit, Sicherheit und Performance).

→ später sind Änderungen schwer übernehmbar

Visualisierung:

- Unified Modeling Language (UML)
- Fundamental Modeling Concepts (FMC)

Eine gute Softwarearchitektur ändert sich nur gering bei einer Änderung eines Problems und macht es Softwareentwicklern leichter, Probleme zu lösen.

Strukturiertes Design (SD)

- legt die technischen Randbedingungen und die technische Grobstruktur fest
- daraus folgt die inhaltliche Planung der Implementierung
- legt die Hierarchie der Module (Bausteine) fest

Objektorientiertes Design (OOD)

Beim objektorientierten Design wird das in der Analyse erstellte Domänenmodell weiterentwickelt und darauf aufbauend ein Systementwurf erstellt. Dabei wird das allgemeine Modell in eine konkrete Softwarearchitektur umgeformt, die Informationen über Details der technischen Umsetzung enthält und direkt als Vorlage für die Implementierung in einer Programmiersprache dient.

Die Unified Modeling Language (*Vereinheitlichte Modellierungssprache*, kurz UML)

Die Unified Modeling Language (Vereinheitlichte Modellierungssprache), kurz UML, ist eine graphische

Modellierungssprache zur Spezifikation, Konstruktion und Dokumentation von Software-Teilen und anderen Systemen. Im Sinne einer Sprache definiert UML dabei Bezeichner für die meisten bei einer Modellierung wichtigen Begriffe und legt mögliche Beziehungen zwischen diesen Begriffen fest. UML definiert weiter graphische Notationen für diese Begriffe und für Modelle statischer Strukturen und dynamischer Abläufe, die man mit diesen Begriffen formulieren kann.

Fundamental Modeling Concepts (FMC)

Fundamental Modeling Concepts (FMC) sind eine semi-formale Methodik zur Kommunikation über komplexe Softwaresysteme.

Nach FMC gibt es drei miteinander verwobene Arten, Softwaresysteme zu betrachten:

- Aufbau des Systems
- Abläufe im System
- Wertebereiche

Für jede dieser Betrachtungsweisen gibt es einen Diagrammtyp, mit dessen Hilfe der jeweilige Aspekt zeichnerisch dargestellt werden kann. Die resultierenden Diagramme sind meist leicht zu erstellen, aber auch leicht zu verstehen.

Grundsätzlich dienen FMC-Diagramme einem von zwei Zwecken: Sie sollen entweder von einer Gruppe verwendet werden, um über ein Softwaresystem zu kommunizieren, oder sie werden benutzt, um andere (Entwickler, Kunden, Manager etc.) in ein Softwaresystem einzuführen. Im ersten Falle werden die Diagramme meist etwas umfangreicher, um die Kommunikation über komplexere Zusammenhänge zu erleichtern; im zweiten Fall werden aus didaktischen Gründen zumeist kleine Diagramme mit wenigen Komponenten verwendet.

4. Programmierung

Allgemein

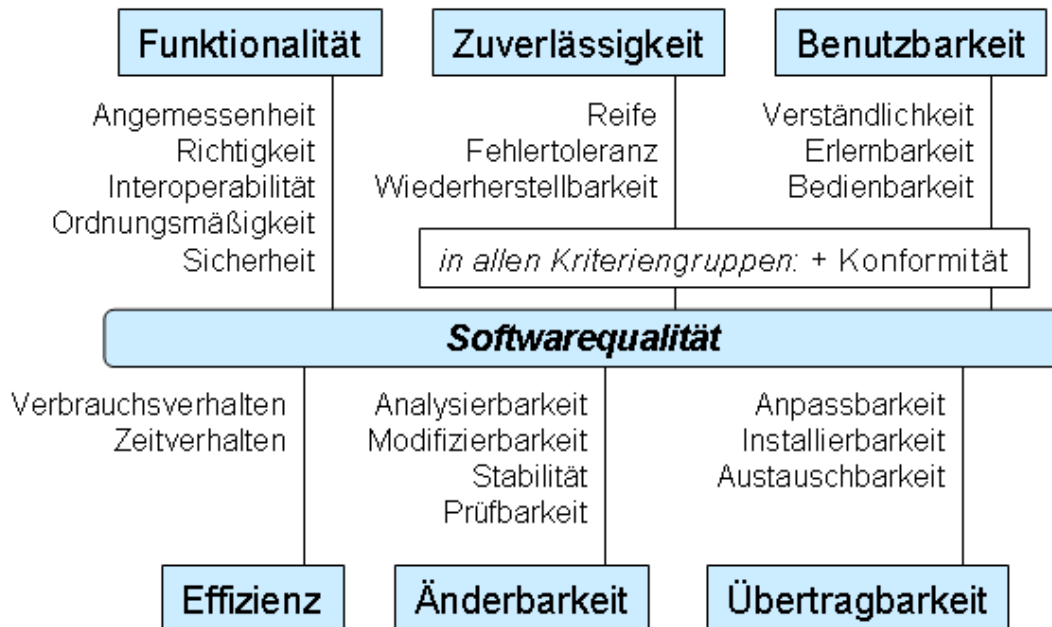
Unter Programmierung versteht man die Umsetzung eines Softwareentwurfes in einen Quellcode. Für die Programmierung werden Programmiersprachen wie beispielsweise [Visual Basic](#) verwendet.

Folgende Qualitätskriterien sollten beim Programmieren beachtet werden:

1. Korrektheit
Das Programm muss den Vorgaben des Softwareentwurfs entsprechen und sollte weitgehend keine Laufzeitprobleme oder syntaktische Fehler beinhalten.
2. Robustheit
Ein Programm wird als robust bezeichnet, wenn es bei nicht vorhersehbaren Fehlern oder einer nicht vorgesehenen Umgebung angemessen reagiert.
3. Wartbarkeit
Beim Programmieren sollte darauf geachtet werden, dass der Quelltext für Dritte leicht verständlich und intuitiv aufgebaut ist. Dies hat den Vorteil, dass ein Dritter bei späteren Änderung es leichter hat den Quelltext zu verstehen.
4. Performanz

Bei der Performanz soll darauf geachtet werden, dass das Programm später ressourcen-sparend ist.

Qualitätsmerkmale von Softwaresystemen (ISO 9126)



Programmierung

- **Normierte Programmierung**

Unter der normierten Programmierung versteht man eine alte, damals nach DIN 66220 genormte Struktur zur allgemeinen Beschreibung von Abläufen. ([Wikipedia Artikel](#)) Entwickelte sich zur Basis der strukturierten Programmierung.

- **Strukturierte Programmierung**

Strukturierte Programmierung beschreibt das Prinzip ein Programm in Teilprogrammen zu zerlegen. Dabei gibt es auf der untersten Ebene drei Kontrollstrukturen:

- Sequenz : hintereinander auszuführende Programmanweisungen
- Auswahl : Verzweigungen (if-Abfragen)
- Wiederholung(Schleifen)

Die strukturierte Programmierung dient als Basis für die funktionale und objektorientierte Programmierung.

- **Objektorientierte Programmierung**: Unter objektorientierter Programmierung versteht man das Prinzip Daten und Funktionen in Objekte einzukapseln und sie somit von anderen Methoden anderer Objekte abzuschirmen.

Folgende Sprachen unterstützen beispielsweise OOP: Java, C++, C#, Visual Basic.

Quellcodebeispiel zu C++ (Definition einer Klasse):

```
HelloWorld.h:  
#include <string>  
class HelloWorld{
```

```

        void setText(string s);
        void printText();
    private:
        string text;
};
HelloWorld.cpp:
#include <iostream>
#include "HelloWorld.h"
using namespace std;
void HelloWorld::setText(string s){
    text = s;
}
void HelloWorld::printText(){
    cout << text;
}

```

- **Funktionale Programmierung**

Programme werden ausschließlich mit Funktionen geschrieben.

Beispiele: Lisp, Scheme, (C/ C++);

Quellcodebeispiel zu Lisp (Definition einer Funktion für die Quadrierung einer Zahl):

```

(defun square (x) (* x x)) ;;Funktionsdefinition:
(princ (square 5)) ;;Ausgabe der Rückgabewertes in der Standardausgabe:

```

5. Validierung und Verifikation

Validierung: dokumentierte Beweisführung, dass ein System die Anforderungen in der Praxis erfüllt

Verifikation: mathematischer Beweis, dass ein Programm (also eine konkrete Implementation) der vorgegebenen Spezifikation entspricht

Eine mögliche Definition (Test):

“Ein Softwaretest [...] ist der überprüfbare und jederzeit wiederholbare Nachweis der Korrektheit eines Softwarebausteines relativ zu vorher festgelegten Anforderungen.”

Globales Ziel: Messen der Qualität des Softwaresystems. Dabei werden definierte Anforderungen überprüft und dabei ggf. vorhandene Fehler aufgedeckt. Der Wirkung von Fehlern (im produktiven Betrieb) wird damit vorgebeugt.

Individuelle Testziele:

Test für jeden einzelnen Testfall, jedes Testobjekt und jede Teststufe

(z. B. Rechenfunktion X in Programm Y getestet, Schnittstellentest erfolgreich, Wiederinbetriebnahme getestet, Lasttest erfolgreich, Programm XYZ getestet usw.)

- Nachweis, dass keine Fehler vorhanden, sind nicht möglich
- nur nachweisbar, dass bestimmte Testfälle erfolgreich waren, da alle Funktionen und alle

möglichen Eingabewerte in allen Kombinationen getestet werden müssten

- → viele Teststrategien beschäftigen sich mit der Frage, wie mit einer möglichst geringen Anzahl von Testfällen eine große Testabdeckung zu erreichen ist

Teststufen

Komponententest (auch Modultest oder Unittest)

Der Komponententest ist ein Test auf der Ebene der einzelnen Module der Software. Testgegenstand ist die Funktionalität innerhalb einzelner abgrenzbarer Teile der Software.

Integrationstest (auch Interaktionstest)

Der Integrationstest testet die Zusammenarbeit voneinander abhängiger Komponenten.

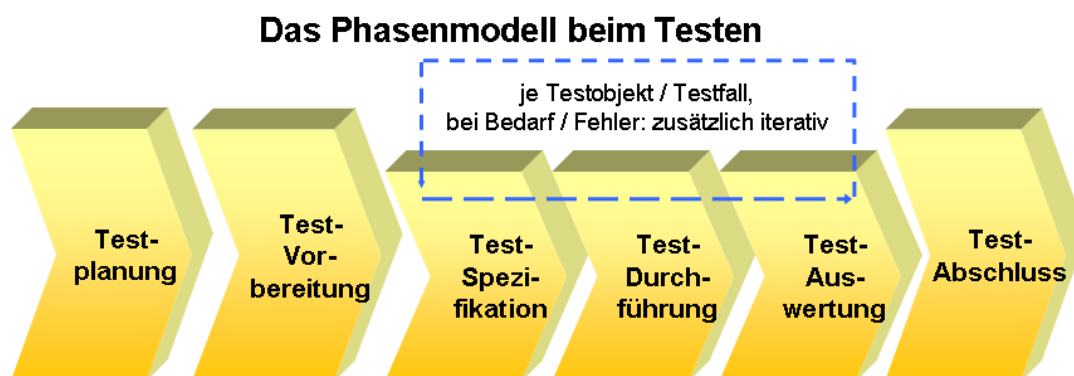
Systemtest

Der Systemtest ist die Teststufe, bei der das gesamte System gegen die gesamten Anforderungen getestet wird. Die Testumgebung soll die Produktivumgebung des Kunden mit Testdaten simulieren.

Abnahmetest (auch Verfahrenstest, Akzeptanztest oder User Acceptance Test (UAT))

Der Abnahmetest ist ein Test der gelieferten Software durch den Kunden bzw. Auftraggeber. Dieser Test kann bereits auf der Produktivumgebung durchgeführt werden.

Testprozess/Testphasen



Quelle: Pol, Koomen, Spillner

Das Modell ist 'generisch', d.h.:

Es wird in mehreren Ebenen, mit unterschiedlichen Schwerpunkten angewendet.

a) Projekt-Ebene, Masterplanung:

b) Je Teststufe:

c) Je Testobjekt / Testfall:

Klassifikation eines Tests:

z.B.

- **Funktionale Tests bzw. Funktionstests**

überprüfen ein System in Bezug auf funktionale Anforderungsmerkmale wie Korrektheit und Vollständigkeit.

- **Oberflächentests**

testen die Benutzerschnittstelle des Systems (z. B. Verständlichkeit, Anordnung von Informationen, Hilfefunktionen)

- **Stresstests**

sind Tests, die das Verhalten eines Systems unter Ausnahmesituationen prüfen und analysieren

- **Crashtests**

sind Stresstests, die versuchen, das System zum Absturz zu bringen

- **Sicherheitstests**

testen ein System gegen potentielle Sicherheitslücken

usw...

6. Anforderungsmanagement

Ziele des Anforderungsmanagements:

Das Ziel des Anforderungsmanagement ist ein gemeinsames Verständnis über eine entwickelnde Software zwischen Arbeitnehmer und Auftraggeber.

Anwendung:

- Lastenheft: Das Lastenheft ist eine Sammlung der Muss-Kriterien des Auftraggebers. Es wird an mögliche Arbeitnehmer gesendet.
- Pflichtenheft: Das Pflichtenheft ist die Antwort der Arbeitnehmer, wo dieser beschreibt, wie er die Probleme im Lastenheft zu lösen gedenkt.

Dies dient zur Qualitätssicherung des Produktes und einer Qualitätssteigerung der Anforderung selbst, da diese Akzeptanzkriterien zur inhaltlichen Überprüfung der Anforderungen führt.

Normen und Sprache:

Anforderungsmanagement verwendet zur Darstellung die natürliche Sprache und verstärkt die künstliche Sprache der Modellierung wie z.B. UML. Ziel einer Anforderungsspezifikation (u. a. Lastenheft, Pflichtenheft,...) ist es, ein gemeinsames Verständnis aller Beteiligten zu erreichen. Ungenaue Verben und Adjektive dürfen nicht verwendet werden. In Deutschland hat sich als Standard das sogenannte Requirements Interchange Format (es wird als XML-Schema definiert) eingebürgert.

Anforderungsmanagement-Software:

Das **Requirements Management System** ist ein Anwendungsprogramm, welches Anforderungen verwaltet. Dies schafft die Voraussetzungen für drei Vorteile eines Requirements Management System gegenüber einer herkömmlichen Ablage von Anforderungen:

1. Anforderungen können verlinkt werden
2. Attribute können hinzugefügt werden
3. Änderungsstände können angezeigt werden

Dies kann grundsätzlich in 2 Einsatzgebiete eingeteilt werden:

- Verwaltung von Anforderungen im Kontext der [Anforderungserhebung](#) (engl. Requirements engineering) und
- Verwaltung von Anforderungen während des [Lebenszyklus einer Anwendung](#) (inklusive [Rückverfolgbarkeit](#))

Es gibt zwei grundlegende Philosophien bei dem Requirements Management: Datenbank-Orientiert und Objekt-Orientiert.

1. Datenbank-orientiert: Die Anforderungen werden als Einzelidentitäten in einer Datenbank verwaltet. Dieser Ansatz ist formal sauber, aber wenig intuitiv.
2. Objekt-orientiert: Die Anforderungen werden, ähnlich wie in einem Objekt der Textverarbeitung, in einer Liste gespeichert.

7. Projektmanagement

Unter dem Begriff Projektmanagement versteht man das Planen, Steuern und Kontrollieren von Projekten.

Der Projektmanager hat die Aufgabe, die Erwartungen der Stakeholder an das Projekt soweit wie möglich zu erfüllen. Der Projektmanager bewegt sich dabei zwischen den Größen Zeit, Kosten, Umfang und Qualität des Projekts. Außerdem bestimmt er über die personelle Besetzung und die Vorgehensweise.

Werden mehrere Projekte gleichzeitig gesteuert und koordiniert, spricht man von Multiprojektmanagement. Vom Multiprojektmanagement abzugrenzen ist der Begriff des Programmmanagements. Unter einem Programm versteht man in diesem Fall ein Bündel inhaltlich zusammengehörender Projekte. Im Projektportfoliomanagement werden die Projekte eines Unternehmens verwaltet. Nahezu alle Teilbereiche des Projektmanagements werden heutzutage durch Projektmanagementsoftware unterstützt.

Zum Projektmanagement gehört auch das Risikomanagement. Risikomanagement ist die systematische Erfassung und Bewertung von Risiken sowie die Steuerung von Reaktionen auf festgestellte Risiken.

8. Qualitätsmanagement

Allgemein: QM bezeichnet alle organisierten Maßnahmen, die der Verbesserung von Produkten, Prozessen oder Leistungen jeglicher Art dienen. Das Ziel ist, die Effektivität und Effizienz zu erhöhen. Die Qualität ist definiert als *das Maß, in dem das betrachtete Produkt oder der betrachtete Prozess den Anforderungen genügt*. Nicht zwangsläufig führt QM zu einem höherwertigen Ergebnis, sondern es steuert lediglich die Erreichung der vorgegebenen Qualität.

Bestandteile im allgemeinen Regelkreis:

Da die kontinuierliche Verbesserung der Prozesse wichtig ist, fließen Erfahrungen daraus wieder in die Planung, ein Regelkreis entsteht.

- **Qualitätsplanung:** es wird ein Ist-Zustand ermittelt und die Rahmenbedingungen für das Qualitätsmanagement festgelegt. Danach werden Konzepte und Abläufe erarbeitet.
- **Qualitätslenkung:** die in der Planphase gewonnenen Ergebnisse werden umgesetzt.
- **Qualitätssicherung:** Auswerten qualitativer und quantitativer Qualitätsinformationen (Kosten-Nutzen-Betrachtungen, Überprüfen von gemachten Annahmen).
- **Qualitätsgewinn:** aus vorheriger Phase gewonnene Informationen werden für Strukturverbesserungsmaßnahmen und Prozessoptimierung eingesetzt. Erfolge und Ergebnisse werden kommuniziert.

Capability Maturity Model

Das Capability Maturity Model (kurz CMM) ist ein Reifegradmodell zur Beurteilung der Qualität ("Reife") des Softwareprozesses von Organisationen sowie zur Bestimmung der Maßnahmen zur Verbesserung desselben.

Aufbau des Modells

Bei CMM wird die Qualität mit einer von fünf Stufen bewertet, wobei die Qualität mit jeder Stufe steigt. Jeder Stufe sind spezielle Key Process Areas (KPAs) zugeordnet, die wiederum Ziele enthalten und beispielhafte Aktivitäten, wie diese Ziele zu erreichen sind. Die Stufen sind:

1. **Initial (beginnend):** Dies ist der Grundzustand, den jede Organisation erreicht, auch ohne dass ein Prozess für die Softwareentwicklung definiert und umgesetzt wird. Kosten, Zeiten und Qualität sind nicht vorhersehbar. Es sind keine KPAs definiert.
2. **Repeatable (wiederholbar):** Ein grundlegender Prozess existiert. Die Planung neuer Projekte erfolgt anhand der Erfahrungen mit vergangenen Projekten. Zeiten sind einigermaßen kontrollierbar. Kosten und Qualität unterliegen starken Schwankungen. Beispiel-KPAs: Anforderungsmanagement, Software-Qualitätssicherung, Softwarekonfigurationsmanagement.
3. **Defined (definiert):** In der Organisation ist ein typischer Software-Entwicklungs- und -wartungsprozess eingeführt und dokumentiert (Standard-Software-Prozess). Eine spezielle Organisationseinheit ist für die Umsetzung verantwortlich. Kosten und Zeit sind hier einigermaßen zuverlässig bewertbar. Qualität ist immer noch Schwankungen ausgesetzt. Beispiel-KPAs: Organisatorische Prozessdefinition, integriertes Softwaremanagement, Begutachtung/Kollegenkontrolle.
4. **Managed (gesteuert):** Sowohl für das Produkt also auch für den Prozess werden quantitative Ziele vorgegeben, ihre Erreichung gemessen und überwacht. Zeiten, Kosten und Qualität sind zuverlässig kontrollierbar. KPAs: Quantitatives Prozessmanagement, Software-Qualitätsmanagement.

- 5. Optimizing (optimierend):** Die gesamte Organisation konzentriert sich auf das Finden von Schwächen und die weitere Verbesserung des Prozesses. KPAs: Defektprävention, Technologieveränderungsmanagement, Prozess- veränderungsmanagement.

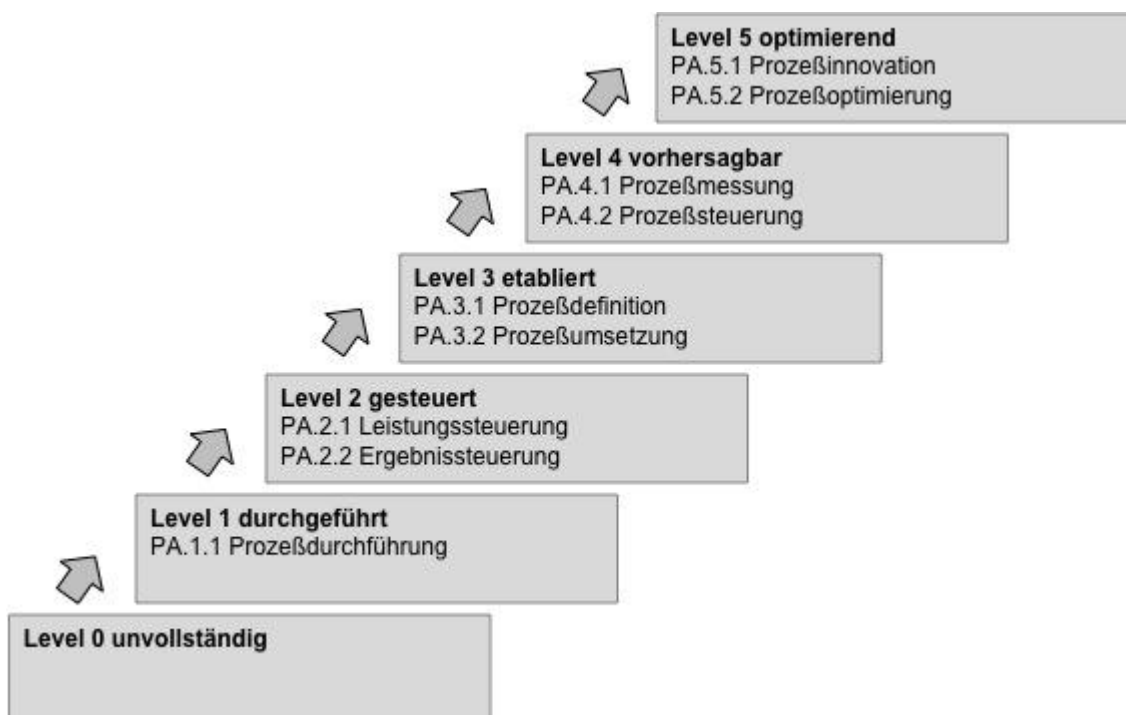
Spice (Norm)

SPICE (*Software Process Improvement and Capability Determination*) ist ein internationaler Standard zur Durchführung von Bewertungen (Assessments) von Unternehmensprozessen mit Schwerpunkt auf der Softwareentwicklung. Die Verbesserung von Prozessen der eigenen Organisation (*Process Improvement*) einerseits und die Bestimmung der Prozessfähigkeit von Lieferanten (*Capability Determination*) andererseits bilden die Kernpunkte dieser Norm. Als Prozess werden hier „Zusammenhängende Aktivitäten zur Transformation von Eingangsprodukten in Ausgangsprodukte“ definiert. Prozess-Assessments werden anhand des zweidimensionalen Referenz- und Assessment-Modells durchgeführt. Die „Prozess-Dimension“ auf der einen Seite dient zur Kennzeichnung und Auswahl der im Assessment zu untersuchenden Prozesse, die „Reifegrad-Dimension“ auf der anderen Seite dient der Bestimmung und Bewertung ihrer jeweiligen Leistungsfähigkeit.

Prozess-Dimension

Der Internationale Standard (IS) formuliert Anforderungen an *Prozessreferenzmodelle* (PRM), mit denen Prozesse beschrieben werden, sowie an *Prozessassessmentmodelle* (PAM), die darauf aufbauend Bewertungskriterien und -methoden für diese Prozesse enthalten. Auf diese Weise werden weder verbindliche Prozesse definiert, noch die Bewertungskriterien dafür festgeschrieben, sondern lediglich grundlegende Anforderungen an solche Modelle beschrieben.

Reifegrad-Dimension



Reifegrade und Prozessattribute

Die 6 Reifegrade treffen Aussagen über die Leistungsfähigkeit der in der Prozess-Dimension beschriebenen Prozesse. Den einzelnen Stufen sind die Aktivitäten zugeordnet, die dazu führen, dass die Ergebnisse systematisch erarbeitet und am Ende des Prozesses in der definierten Qualität vorliegen. Den Reifegradstufen sind insgesamt neun Prozessattribute zugeordnet. Diese werden jeweils durch die ihnen zugeordneten grundlegenden Managementaktivitäten beschrieben und dienen der Beurteilung der Prozesse. Der Reifegrad wird für jeden Prozess einzeln bestimmt. Es wird nicht nur die Existenz einer Prozessaktivität beurteilt, sondern auch die adäquate Durchführung der Aktivität bewertet. Die Bewertung jedes Prozessattributs erfolgt anhand einer vierstufigen Skala („nicht erfüllt“: 0% - 15%, „teilweise erfüllt“: >15% - 50%, „weitgehend erfüllt“: >50% - 85%, „vollständig erfüllt“: >85% - 100%).

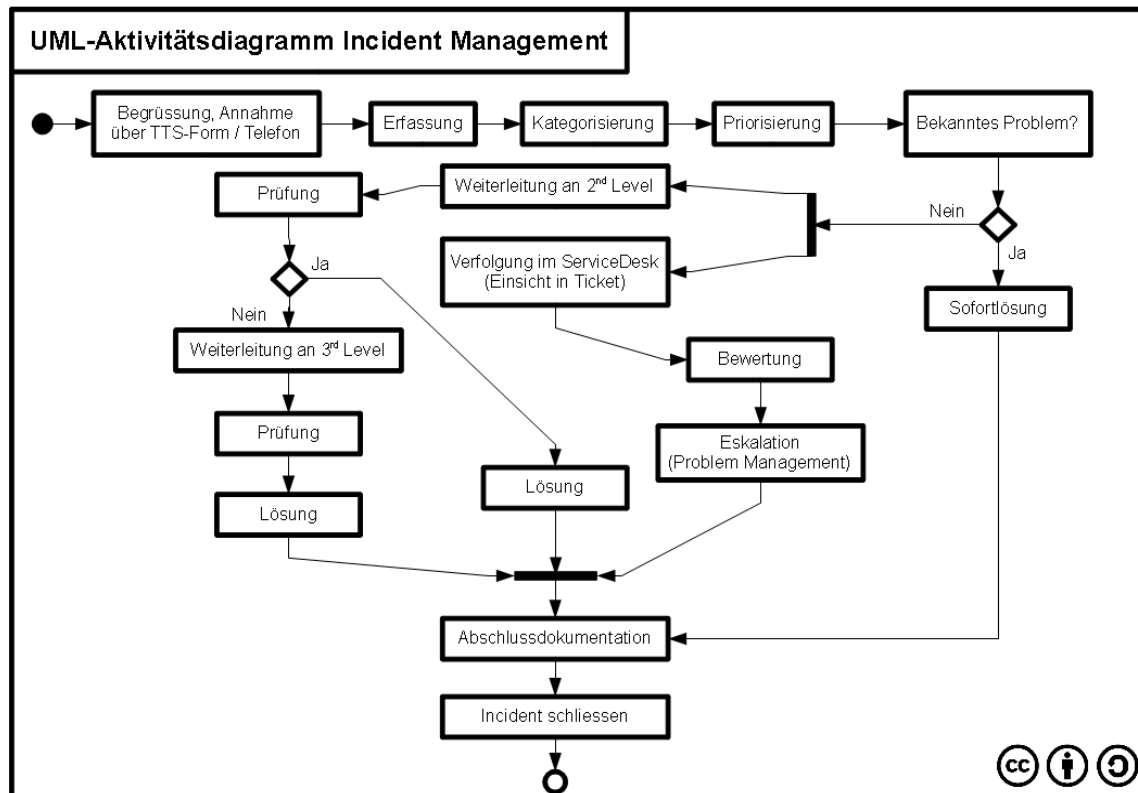
Für die Auswertung des Assessments werden die Prozess- und die Reifegraddimension zusammengeführt. Hierbei werden für jeden untersuchten Prozess Bewertungen in Form der Erfüllungsgrade der neun Prozessattribute ermittelt, woraus sich der Reifegrad für diesen Prozess ergibt. In der Gesamtheit aller untersuchten Prozesse ergibt sich daraus ein Stärken-Schwächen-Profil, aus dem Verbesserungspotenziale erkennbar werden. Die Beschreibungen des jeweils nächsthöheren Reifegrades zeigen Möglichkeiten zur Verbesserung der Prozesse auf.

Incident Management

IT-Incident Management umfasst typischerweise den gesamten organisatorischen und technischen Prozess der Reaktion auf erkannte oder vermutete Sicherheitsvorfälle bzw. Störungen in IT-Bereichen sowie hierzu vorbereitende Maßnahmen und Prozesse. Das Spektrum möglicher Vorfälle reicht dabei von technischen Problemen und Schwachstellen bis hin zu konkreten Angriffen auf die IT-Infrastruktur. IT-Incident Management im engeren Sinne muss dabei sowohl organisatorische, als auch rechtliche sowie technische Detailfragen berücksichtigen.

Ziel des Incident-Management-Prozesses ist die schnellstmögliche Wiederherstellung der Service-Leistung.

Unter einem *Incident/Störung* versteht man nach IT Infrastructure Library (ITIL): "Ein Ereignis, das nicht zum standardmäßigen Betrieb eines Services gehört und das tatsächlich oder potenziell eine Unterbrechung dieses Services oder eine Minderung der vereinbarten Qualität verursacht."



Problem-Management

Über das Problem-Management werden unbekannte Ursachen für tatsächliche und potentielle Störungen (Incidents) innerhalb der IT-Services untersucht und die Behebung gesteuert. Anders als das Incident Management arbeitet das Problem-Management sowohl reaktiv als auch proaktiv. Ein wesentliches Ziel ist hierbei die 'dauerhafte Problemlösung'.

Problem-Management analysiert mögliche oder bereits eingetretene Störungen und identifiziert daraus Probleme, die genauer untersucht werden sollen. Die Ursachen werden analysiert und Maßnahmen zu ihrer Verhinderung oder Behebung entwickelt. Ergebnis dieser Analyse ist entweder ein *Known-Error* (also die nun bekannte Ursache für eine Störung) oder eine Umgehungslösung (*Workaround*).

Im Rahmen von Softwaretests wird ein auf diese Aufgabenstellung speziell ausgerichtetes Problem-Management, dort auch Fehlermanagement, Bugfixing etc. genannt, betrieben.

Aufgaben

- Die Ursache für Betriebsstörungen (Probleme) untersuchen
- Probleme dauerhaft beheben und Umgehungslösungen entwickeln
- Sicherung des Wissens über bekannte Problemursachen
- frühzeitiges Erkennen möglicher Probleme, bevor diese auftreten

Ziele

- Senkung der Störungsanfälligkeit und nachhaltige Erhöhung der Servicequalität
- Erkennen von Beziehungen zwischen Störungen, gemeinsame Ursachen finden

Aktivitäten

- Problem Control (Problembehandlung)
- Error Control (Fehlerbehandlung)
- Proaktives Problemmanagement

Nutzen

- signifikante Reduktion der auftretenden Störungen
- Verbesserung der Zuverlässigkeit der IT Services
- dauerhafte Problemlösung
- Senkung der Qualitätskosten

Softwaremetrik

Eine Softwaremetrik, oder kurz Metrik, ist eine (meist mathematische) Funktion, die eine Eigenschaft von Software in einen Zahlenwert, auch Maßzahl genannt, abbildet. Hierdurch werden formale Vergleichs- und Bewertungsmöglichkeiten geschaffen.

Einfache Metriken zeigen die Größe des Quellcodes in Zeilen oder Zeichen auf, komplexere Metriken versuchen die Verständlichkeit des Quellcodes zu beurteilen. Mit einer geeigneten Zahl verschiedener Metriken kann beurteilt werden, wie aufwändig (sprich personal- und kostenintensiv) die Wartung, Weiterentwicklung und anschließende Tests der Software werden. Werden während der langfristigen Weiterentwicklung einer Software regelmäßig Metriken angewendet, können negative Trends, also Abweichungen vom Qualitätsziel, frühzeitig entdeckt und korrigiert werden.

Ordnung von Softwaremetriken

Nutzung

Der Einsatz von Metriken erstreckt sich von der Beurteilung der Entwicklungsphasen über die Beurteilung der Phasenergebnisse bis hin zur Beurteilung der eingesetzten Technologien. Das Ziel der Anwendung einer Metrik in der Softwareentwicklung ist die Fehlerprognose und die Aufwandschätzung, wobei zwischen vorlaufendem, mitlaufendem und retrospektivem Einsatz unterschieden wird.

Beschränkung

Grundsätzlich sind Metriken, die überschaubar bleiben, eindimensional. Damit zwingen sie zur Vereinfachung. In der Regel wird das erreicht, indem jede Metrik auf eine Sicht eingeengt wird. Das bedeutet dann zwingend, dass andere Sichten nicht gleichzeitig in gleicher Qualität bedient werden.

1. Sicht des *Managements*

- Kosten der Software-Entwicklung (Angebot, Kostenminimierung)
- Produktivitätssteigerung (Prozesse, Erfahrungskurve)
- Risiken (Marktposition, Time2Market)
- Zertifizierung (Marketing)

2. Sicht des *Entwicklers*

- Lesbarkeit (Wartung, Wiederverwendung)
- Effizienz und Effektivität
- Vertrauen (Restfehler, MTBF, Tests)

3. Sicht des *Kunden*

- Abschätzungen (Budgettreue, Termintreue)
- Qualität (Zuverlässigkeit, Korrektheit)
- Return on Investment (Wartbarkeit, Erweiterbarkeit)

Klassifikation

Für die verschiedenen Aspekte der Bewertung gibt es *Entwurfsmetriken*, *wirtschaftliche Metriken*, *Kommunikationsmetriken* usw. Metriken können verschiedenen Klassen zugeordnet werden, die den Gegenstand der Messung oder Bewertung bezeichnen:

1. *Prozess-Metrik*

- Ressourcenaufwand (Mitarbeiter, Zeit, Kosten)
- Fehler
- Kommunikationsaufwand

2. *Produkt-Metrik*

- Umfang (Lines of Code, Wiederverwendung, Prozeduren, ...)
- Komplexität
- Lesbarkeit (Stil)
- Entwurfsqualität (Modularität, Kohäsion, Kopplung, ...)
- Produktqualität (Testergebnisse, Testabdeckung, ...)

3. *Aufwands-Metrik*

- Aufwandsstabilität
- Aufwandsverteilung
- Produktivität
- Aufwand-Termin-Treue

4. *Projektlaufzeit-Metrik*

- Entwicklungszeit
- Durchschnittliche Entwicklungszeit
- Meilenstein-Trend-Analyse
- Termintreue

5. Komplexitäts-Metrik

- Softwaregröße
- Fertigstellungsgrad

6. Anwendungs-Metrik

- Schulungsaufwand
- Kundenzufriedenheit

Gütekriterien

Eine Metrik aus der Produktionsphase der Software allein ist noch kein Gütekriterium. In der Regel werden Güteermale an der Erfüllung der Anforderungen des Kunden und seiner Anwendung gemessen. Dabei sind die Übertragbarkeit der Ergebnisse und die Repräsentanz der Messwerte für den Kundennutzen von Bedeutung:

- *Objektivität*: keine subjektiven Einflüsse des Messenden
- *Zuverlässigkeit*: bei Wiederholung gleiche Ergebnisse
- *Normierung*: Messergebnisskala und Vergleichbarkeitsskala
- *Vergleichbarkeit*: Maß mit anderen Maßen in Relation setzbar
- *Ökonomie*: minimale Kosten
- *Nützlichkeit*: messbare Erfüllung praktischer Bedürfnisse
- *Validität*: von messbaren Größen auf andere Kenngrößen zu schließen (schwierig)

Statische Analyse

Der Quelltext wird einer Reihe formaler Prüfungen unterzogen, bei denen bestimmte Sorten von Fehlern entdeckt werden können, noch bevor die entsprechende Software (z. B. im Modultest) ausgeführt wird. Die Methodik gehört zu den falsifizierenden Verfahren, d. h. es wird die Anwesenheit von Fehlern bestimmt. Man spricht von *statischer* Analyse, da die zu testende Software in Form von Algorithmen und Daten in ihrer Formulierung und Beschaffenheit (*statisch*) dem Prüfer (oder Werkzeug) vorliegt.

Software-Ergonomie

SE ist die Arbeit hin zu *leicht verständlicher* und *schnell benutzbarer* Software unter den *gebotenen technischen Möglichkeiten* und unter der Einhaltung definierter bzw. empirisch entstandener Standards und Styleguides. Das Ergebnis ist die Gebrauchstauglichkeit, also die Benutzerfreundlichkeit von Computer-Programmen.

Gegenstandsbereich der SE im eigentlichen Sinne ist der arbeitende Mensch (Softwarenutzung an Arbeitsplätzen) und die zu optimierende Softwarenutzung an Arbeitsplätzen. Allgemein wird heute die Benutzung von, beziehungsweise die Interaktion mit Computern betrachtet. Dies bedeutet die Berücksichtigung (neuro-)psychologischer Aspekte.

Allgemein befasst sie sich mit Grundregeln und Methoden zum Entwurf und zur Bewertung von interaktiver Software (wie Arbeitssoftware, WWW, Spiele usw.), die möglichst optimal an die

Bedürfnisse der Benutzer (Benutzerorientierung) und die Erfordernisse der Arbeitsaufgabe (Aufgabenorientierung) anzupassen ist. Die Belastungsminderung und Handlungsunterstützung durch das System stehen im Vordergrund.

Kriterien

Als Grundlage für die Definition ergonomischer Prinzipien eignet sich das ABC-Modell, das den Nutzungskontext der Ergonomie als Beziehungen zwischen Aufgabe, Benutzer und Computer beschreibt. Dabei wurden 7 Kriterien festgelegt:

1. Aufgabenangemessenheit – geeignete Funktionalität, Minimierung unnötiger Interaktionen
2. Selbstbeschreibungsfähigkeit – Verständlichkeit durch Hilfen / Rückmeldungen
3. Steuerbarkeit – Steuerung des Dialogs durch den Benutzer
4. Erwartungskonformität – Konsistenz, Anpassung an das Benutzermodell
5. Fehlertoleranz – unerkannte Fehler verhindern nicht das Benutzerziel, erkannte Fehler sind leicht zu korrigieren
6. Individualisierbarkeit – Anpassbarkeit an Benutzer und Arbeitskontext
7. Lernförderlichkeit – Minimierung der Erlernzeit, Metaphern, Anleitung des Benutzers

Später wurden diese Kriterien um 4 Grundsätze ergänzt:

1. Eignung für das Kommunikationsziel – verwendete Informationen / Medien unterstützen die intendierten Kommunikationsziele.
2. Eignung für Wahrnehmung und Verständnis – Inhalte sind so aufbereitet, dass sie gut rezipiert werden können und leicht verständlich sind.
3. Eignung für die Exploration – Informationen sollen gut strukturiert sein, so dass die Erkundung der Informationen und das Stöbern in den Informationen leicht ist.
4. Eignung für die Benutzungsmotivation – Das Programm soll zur Benutzung motivieren und eine hohe Bindung des Nutzers erreichen.

9. Konfigurationsmanagement

Konfigurationsmanagement bezeichnet die Anwendung von Regeln auf den Produktlebenslauf

9.1 Versionsverwaltung

- Verwaltungssysteme
 - Abkürzung VCS (**V**ersion **C**ontrol **S**ystem) gebräuchlich
 - CVS (**C**oncurrent **V**ersions **S**ystem) nicht mehr aktiv weiterentwickelt
 - **Git** als freie Software zur verteilten Versionsverwaltung von Dateien auch gebräuchlich
 - **SVN** (Apache Subversion)
 - Versionierung erfolgt in einem zentralen Projektarchiv (Repository)
 - Weiterentwicklung von [CVS](#)
- System zur Erfassung von Änderungen in Dokumenten oder Dateien
- Sicherung mit Zeitstempel und Benutzerkennung
 - dient zur späteren Wiederherstellung
- Nachvollziehen von Änderungen

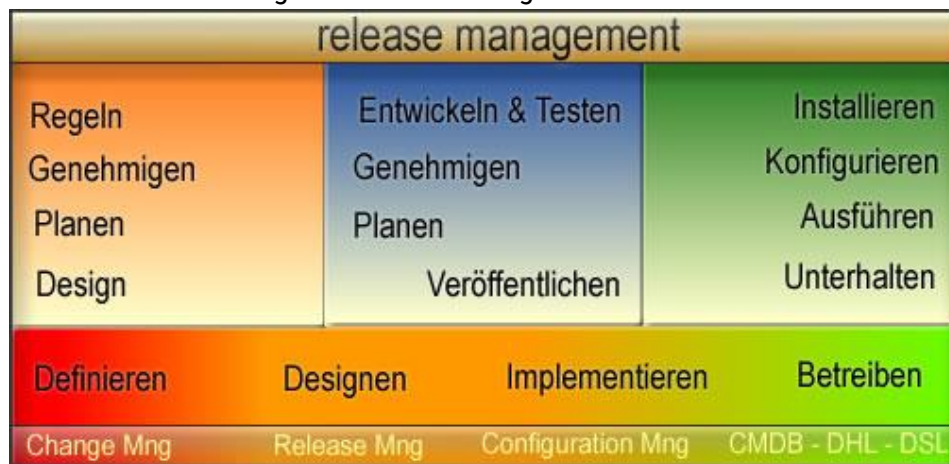
- Gleichzeitige Entwicklung mehrerer Entwicklungszweige

9.2 Veränderungsmanagement

- Zusammenfassung aller Maßnahmen und Tätigkeiten, die eine Veränderung bewirken sollen
- 3 Phasen der Veränderung
 - Auftauphase (unfreezing)
 - Einsicht, dass die Erwartungen nicht mehr der Realität entsprechen
 - Wille zur Veränderung kommt auf
 - Bewegungsphase (moving)
 - Generieren von Lösungen
 - Neue Verhaltensweisen ausprobieren
 - Einfrierphase (refreezing)
 - Implementieren der gefundenen Problemlösungen
 - Vorläufiger Abschluss

9.3 Release Management

- Festlegung des funktionellen Umfangs
- Festlegung des genauen Zeitplans einer Release-Freigabe
- Dokumentation des Umfangs und der Änderungen



9.4 Application Management

- Kombination aus der Entwicklung und Betreuung von Applikationen
- umfassende Anwenderbetreuung (= Support)
- Weiterentwicklung der Software

10. Softwareeinführung

Die Softwareeinführung bedeutet die schrittweise Einführung einer Software in eine größere Organisation. Dazu gehört die Verteilung, Installation und Konfiguration der Software, sowie die Schulung der Mitarbeiter und das Umstellen firmeninterner Prozesse. Außerdem gehört zu dem Prozess der Einführung die Einrichtung einer IT-Betriebsorganisation (Operating).

Es gibt zwei Wege zur Einführung einer Software:

10.1 Big Bang

Die Software wird zu einem klar definierten Zeitpunkt in Betrieb genommen. Dies erfordert eine umfangreiche Planung und Vorbereitung, denn es muss ab dem Zeitpunkt der Einführung alles funktionieren. Dazu gehört, dass die Mitarbeiter (Administratoren, Operating und Endnutzer) geschult sein müssen und die Software umfangreich getestet sein muss. Für den Fall, dass unvorhergesehene Fehler auftreten sollte ein Rollback, die Zurücksetzung auf den vorherigen Zustand, geplant werden. Ein Risiko ist, dass die Software nicht oder nur mit großer Verspätung implementiert wird.

10.2 Iterative Einführung

Die Software wird in mehreren, kleinen Schritten eingeführt. Dazu müssen die einzelnen Abhängigkeiten der Software analysiert werden. Eine Webanwendung kann nicht installiert werden, wenn nicht vorher der Web-Server installiert wurde. Die iterative Einführung soll das Risiko eines kompletten Systemausfalls reduzieren und bietet die Möglichkeit, dass die Nutzer schon frühzeitig Erfahrungen mit der Software sammeln und diese einfließen in den Prozess der Entwicklung. Es gibt deshalb einen langen Zeitraum, in dem mit unvollständigen Zwischenlösungen gearbeitet wird. Wenn es noch ein Altsystem gibt, dann kann sich der Aufwand für die Nutzer verdoppeln, weil Daten in beide System eingegeben werden müssen.

Ein großes Risiko ist die Änderung der internen Firmenpolitik, da dadurch aus Kostengründen das Projekt in einem unfertigen Zustand gestoppt/verkleinert werden kann und im schlimmsten Fall unbrauchbar ist.

11. Dokumentation

11.1 Projektdokumentation

Unter Projektdokumentation versteht man die Dokumentation des Entwicklungsprozesses. Dies kann aus juristischer Sicht wichtig sein.

Typische Punkte, die in ihr festgehalten werden, können sein:

- Anforderungen
- Teamaufteilung
- Entwicklungs-Prozesse

11.2 Softwaredokumentation

Hier wird wiederum unterschieden zwischen Programmierer- und Benutzerdokumentation.

11.2.1 Benutzerdokumentation

Die Benutzerdokumentation liegt meistens in Form eines Benutzerhandbuches vor. Darin enthalten sein sollte zumindest ein Leitfaden zur Benutzung der Software. Außerdem kommen oft FAQs und Fehlerbehandlungsmethoden vor.

11.2.2 Programmiererdokumentation

Die Programmiererdokumentation ist, wie der Name schon sagt, die Dokumentation für Entwickler und

Programmierer. Diese unterteilt sich ihrerseits wieder in einige Bereiche.

11.2.2.1 "Äußere" Dokumentation

Diese Dokumentation richtet sich an Programmierer, die Schnittstellen (API) von diesem Programm nutzen wollen. Sie beinhaltet Funktionsdefinitionen und Rückgabewerte und was diese Funktionen ausrichten.

11.2.2.2 Innerhalb des Quellcodes

Im Quellcode sollten gewisse Richtlinien gelten. Variablennamen sollten beispielsweise sinnvolle und ohne Kommentar verständliche Namen haben. Generell gilt: Wo sich Kommentare durch gut strukturierten Code vermeiden lassen, ist dieser zu wählen.

Kommentare sollten so nah wie möglich an der entsprechenden Code-Stelle stehen.

Prinzipiell sollten Kommentare, Anmerkungen und ähnliches so vollständig wie möglich im Code stehen. Nur, wenn es sich nicht vermeiden lässt, sollten die Dokumente extern gespeichert werden, dann aber immer mit dem Code verteilt werden.

11.2.3 Methodendokumentation

Die Methodendokumentation beschreibt die der Software zugrunde liegenden Prinzipien, also die Algorithmen, nach denen sie arbeitet.

11.2.3.1 Javadoc

Javadoc ist ein Werkzeug um automatisiert aus bestehendem Quellcode HTML-Seiten mit den Methodenspezifikationen zu erstellen. In Netbeans hat das Projektmenü (Rechtsklick auf den Projektordner) einen Eintrag "Generate Javadoc". Wählt man diesen aus, wird ein Ordner */Projektordner/dist/javadoc* erstellt. Wenn man in diesem die Datei "index.html" öffnet, erscheint ein Klassen- und Funktions-Browser. Zusätzlich zu den bloßen Klassen- und Funktionsdefinitionen kann man noch weitere Informationen als Kommentare im Quellcode einfügen. Eine Übersicht über alle Tags findet man auf [Wikipedia](https://www.oracle.com/de/javase/8/docs/javadoc/).

11.3 Testdokumentation

In ihr werden alle Testfälle und deren Resultate dokumentiert, sowie mögliche Testfälle, mit denen man zukünftige Versionen testen sollte.

Anhang

Techniken bei der Nutzung von Zeigern/Objektreferenzen in Java

Will man bspw. einen Zeiger durch z.B. eine Liste laufen lassen, muss man sich einen Hilfszeiger erstellen, der dann vom Kopf aus immer weiter verschoben wird, bis man dort ankommt, wo man hin will.

Beispiel:

```
Knoten hilfszeiger = head;
while(...) {
    [...]
    hilfszeiger = hilfszeiger.next;
    [...]
}
```

Tausch von Elementen

Zum Tauschen von Elementen in einer Liste kann man den sogenannten *Dreieckstausch* verwenden:

```
int temp = hz1.value;    // hz1.value zwischenspeichern
hz1.value = hz2.value;   // hz2.value in hz1.value speichern
hz2.value = temp;        // hz2.value durch temp ersetzen
```

Weitere Beispielaufgaben zur Rekursion

Beispiel: Reverse

Aus "Hallo" soll "ollaH" werden.

```
public String reverse (String text) {
    if (text.equals("")){          // oder: if(text.isEmpty()){
        return "";
    } else {
        int l = text.length();
        String b = text.substring(l-1,l);
        String c = text.substring(0,l-1);
        return b + reverse(c);
    }
}
```

Beispiel: [Fibonacci-Folge](#)

```
public int fib (int stelle){
    if (stelle==1 || stelle==0){
        return 1;
    } else{
        return fib(n-1) + fib(n-2);
    }
}
```

Fibonacci-Folge mit Schleife

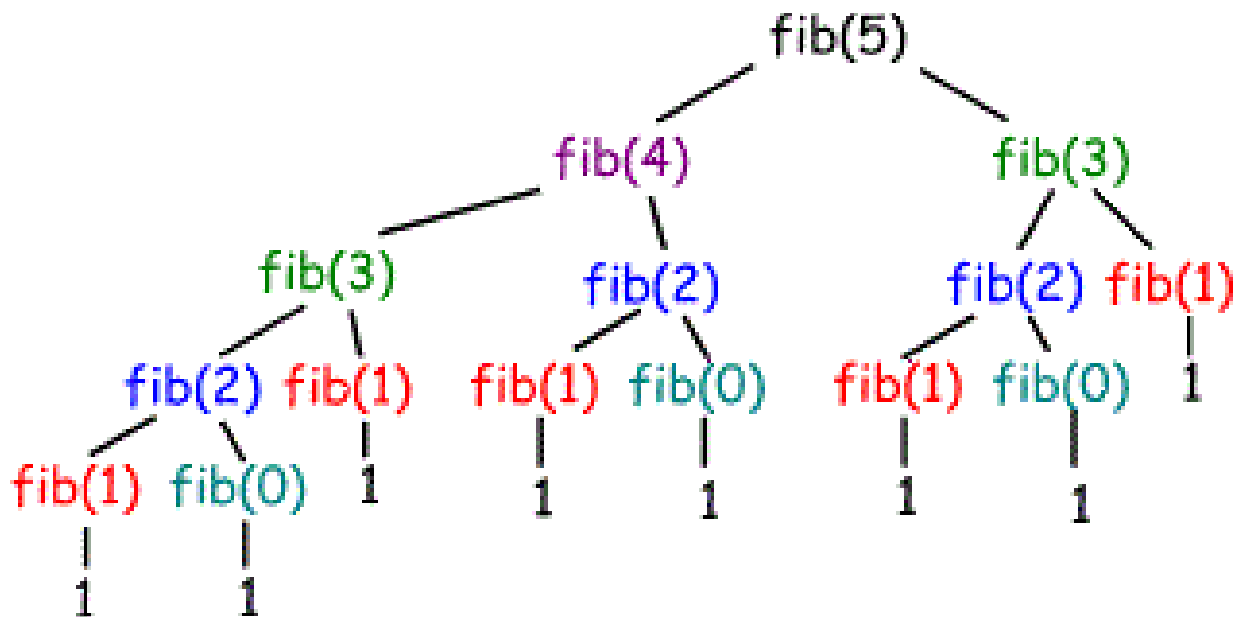
```
public int fib (int stelle){
    if (stelle == 1 || stelle == 0){
        return 1;
    } else {
        int a, b, c, i = 1;
        while (i < stelle) {
            b = a;
            a = c;
            c = a+b;
            i++;
        }
        return c;
    }
}
```

```

    }
}

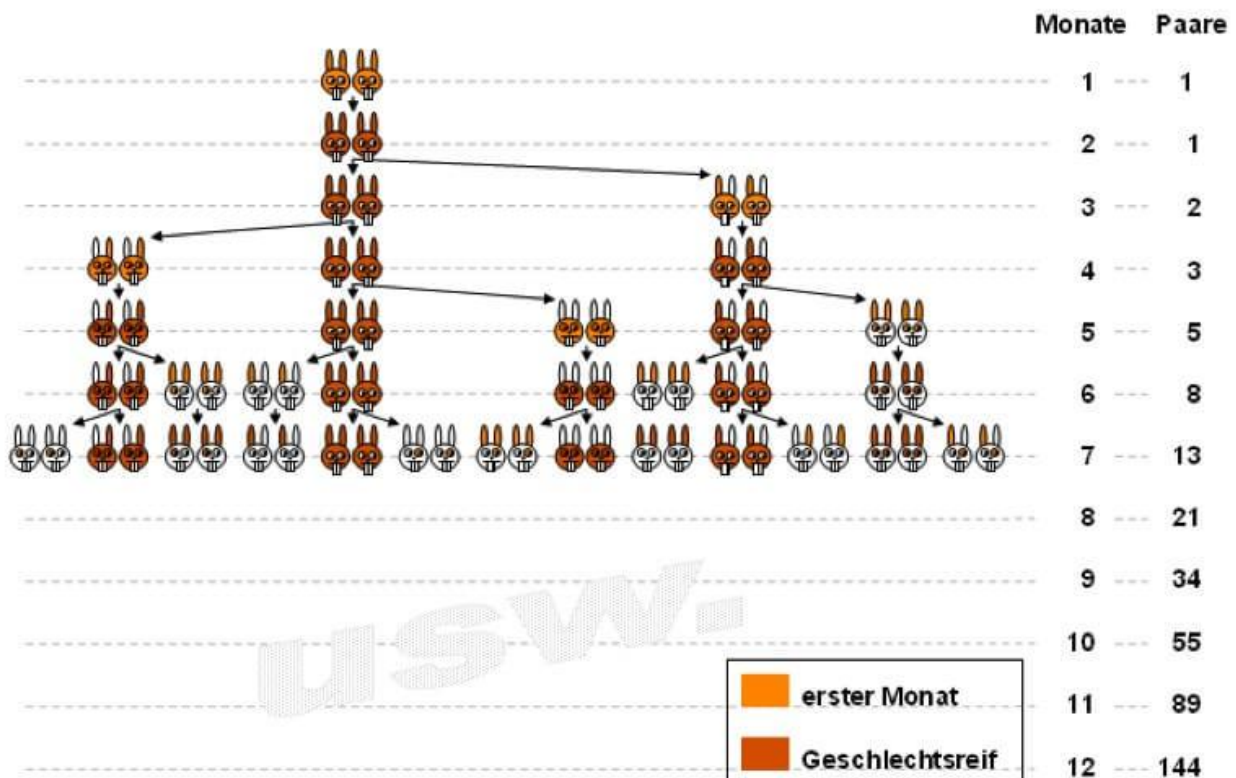
```

Vergleich mit der rekursiven Lösung



Problem: sehr speicherlastig!

Lösung: Zwischenspeichern von Ergebnissen wie fib(2) und fib(3) in einer höheren Datenstruktur wie zum Beispiel der Liste. So müssen diese nicht mehrmals berechnet werden.



Rekursion: Die Türme von Hanoi

Informationen unter: https://de.wikipedia.org/wiki/Türme_von_Hanoi

```
public class Hanoi {  
    public static void hanoi(String von, String ueber, String nach, int n){  
        if(n==1){  
            System.out.println(von + " --> " + nach);  
        } else {  
            hanoi(von, nach, ueber, n-1);  
            System.out.println(von + " --> " + nach);  
            hanoi(ueber, von, nach, n-1);  
        }  
    }  
}
```

Aufruf aus einem Hauptprogramm:

```
Hanoi.hanoi(„A“, „B“, „C“, 6);
```

Dabei stehen A, B und C für die Stäbe und 6 für die Anzahl der Scheiben.